EasySoft
SE Development

SOFTWARE ENGINEERING & DEVELOPMENT · SRI LANKA

foodhub

System Development Report

Version 1.0 web application prototype · Project close-out record

DOCUMENT
ES-FH-SDR-1.0

STATUS
Completed prototype

ISSUE DATE
27 October 2026

DEVELOPER
Evan Keal

ORGANIZATION
EasySoft

PROJECT TYPE
Web application prototype

SYSTEM DESIGN
Manual, diagrams.net (draw.io)

INTERFACE DESIGN
AI-assisted visual design

Designed to evolve. Built to improve.

A completed web ordering prototype, and the design baseline for the FoodHub system that comes next. Diagrams in this report were drawn for the Windows application and its administrative side. They guide future development. They are not a list of features already shipped in version 1.0.

EASYSOFT · © 2026
EVAN KEAL

DOCUMENT CONTROL

Control sheet and contents

Field	Entry
Title	FoodHub Version 1.0 — System Development Report
Document ID / revision	ES-FH-SDR-1.0 · Revision 1.0 · Issued
Author	Evan Keal, developer, EasySoft
Brand	EasySoft — personal software engineering brand
Repository	https://github.com/aimstudios3001/FoodHub-Food-Order-System.git
Portfolio	https://evan-keal-portfolio.vercel.app
Predecessor	FoodHub desktop application, C# and .NET
Design method	Workflow and structure drawn manually in diagrams.net (draw.io). Interface produced with AI-assisted design.
Implementation method	AI-assisted programming in Visual Studio Code, versioned with Git

Rev	Date	Description	Author
1.0	27 Oct 2026	Initial issue at completion of the web prototype	Evan Keal

How to read this report. Sections 1–12 describe the FoodHub 1.0 web prototype as it was actually built: a customer storefront and an administrator desk. From Section 13, every figure is part of the future design baseline. Those drawings were prepared for the FoodHub Windows application and for administrative operations. They are the plan the system will grow into. They are not evidence that riders, vehicles, ingredients, dependents, or dispatch already exist in the web prototype.

Contents

THE COMPLETED PROTOTYPE

1 Executive summary 3
2 Introduction 4
3 Background 5
4 Objectives and scope 6
5 Development methodology 7
6 Technology stack 8
7 Tools, build chain, and repository 9
8 Architecture of version 1.0 10
9 Functional design 11
10 Implemented data model 13
11 Security and sessions 14
12 Service interface, UI, and delivery 15

FUTURE SYSTEM — WINDOWS APPLICATION AND ADMIN DESIGN

13 Design boundary and operating model 18
14 Use cases, data flow, and workflows 19
15 Class model and conceptual data 27
16 Logical schema and normalization 31
17 Gap analysis 40
18 Future development program 41
19 Risks, limitations, and conclusion 43

SECTION 1

Executive summary

FoodHub 1.0 is a completed web application prototype from EasySoft. It takes the food-ordering idea of an earlier C# .NET desktop system and rebuilds a usable slice of it as a browser application: customers browse a menu and place orders; an administrator runs the catalog, the order board, and accounts.

1.0	2 roles	3 tiers	2026
COMPLETED VERSION	CUSTOMER AND ADMIN	VUE, C++, SQLITE	EASYSOFT · EVAN KEAL

The prototype is deliberately small in structure and serious in the path it proves. One Vue single-file application talks to one C++ process. That process, built with CMake and the Crow HTTP library, stores data in SQLite and protects passwords with OpenSSL. Money is kept as integer minor units of Sri Lankan rupees. Placing an order is a single database transaction that checks stock, snapshots prices, and clears the cart together.

What version 1.0 is not: it is not the full FoodHub operating model. The use-case diagram, data-flow diagrams, flowcharts, class diagram, and the 1NF, 2NF, and 3NF logical models were drawn by hand in diagrams.net for the Windows application and for the administrative side of that system. They introduce staff, riders, vehicles, meter readings, dependents, ingredients, and formal reports. None of those operational actors are implemented in the web prototype. This report keeps the two layers apart on purpose, then uses the drawings as a sequenced plan for what EasySoft builds next.

What was completed

- Account registration and sign-in, with a separate administrator gate.
- A customer menu with search, categories, cart, checkout, order history, profile, and notifications.
- An administrator desk for food items and images, order status, user activation, and a monthly order view.
- A documented Windows workstation build: Visual Studio 2026 generator, vcpkg, CMake, and a Vite front end.
- Publication of the source as the first commit on the project repository.

What the next system must still become

The next releases should harden the prototype before they widen it: split the single C++ file and the single Vue file, add migrations and tests, and stop treating a public registration form as the way an administrator is created. Only then should the desktop design be brought across — staff-entered orders, ingredient transparency, rider assignment, vehicle and meter records, delivery failure, and administrator reports. Section 18 sets that sequence. Section 16 records the modeling corrections required so the 3NF drawings become a physical schema without copying known diagram defects into code.

> **Project position.** Version 1.0 is closed as a prototype. The improvements in this report are planned. They must not be read as features already delivered.

SECTION 2

Introduction

Purpose

This report is the system development record for FoodHub Version 1.0. It explains why the prototype exists, which technologies and tools produced it, how the running system is structured, and how the hand-drawn design set should drive later versions. It is written for a reader who needs a professional account of the work: the developer's own archive, a portfolio reader, or a reviewer of the EasySoft project.

Audience

- Evan Keal, as the author of record and the person who will extend the system.
- Readers of the EasySoft portfolio who need more than a repository README.
- A future collaborator who must not confuse the web prototype with the Windows design.

System identity

Item	Statement
Name	FoodHub Food Order, Version 1.0
Kind	Full-stack web application prototype
Outcome	Completed. Not claimed as a hosted production service.
Developer	Evan Keal
Brand	EasySoft, an independent software engineering brand focused on practical systems and modern tools
Country context	Sri Lanka. Prices are displayed in LKR. The seeded menu includes Rice & Curry alongside broader categories.
Public code	github.com/aimstudios3001/FoodHub-Food-Order-System
Portfolio	evan-keal-portfolio.vercel.app

Sources used to write this record

The report is grounded in the prototype source, not in a generic food-ordering template. The backend is `backend/main.cpp` with `backend/CMakeLists.txt` and `vcpkg.json`. The interface is `frontend/src/App.vue` and `frontend/src/style.css`. The workstation runbook is `Run System info.txt`. The narrative README states the same project facts this report expands. The figures are the original draw.io exports supplied with the design set: use case, two data-flow levels, four workflows, a class diagram, a conceptual ERD, two logical schema drawings, and the 1NF, 2NF, and 3NF logical models.

Conventions

Identifiers from the drawings (CUST_ID, ORD_NO, and the rest) are kept in capitals when the report is talking about those drawings. Identifiers from the prototype (users, foods, price_minor) are kept as implemented. "Staff" in the drawings is not the same role as "admin" in the prototype. Where a drawing contains a spelling that should not be copied into a database — ITEM_CATOGORY, DispatchLacotion, "Seve" for Save — the report says so once and moves on.

SECTION 3

Background

FoodHub began as a desktop application written with C# and .NET. That system was modeled as an operation, not only as a shop window. A customer places an order. A staff member enters and maintains order detail. A rider is assigned, takes a vehicle, records a motorbike meter, and updates delivery. A system administrator maintains customers, items, ingredients, riders, dependents, vehicles, and reports. The draw.io set in the later sections of this report is that model.

EasySoft then asked a different engineering question: can the same business idea live as a web application, with a modern browser interface, a C++ service, and a file-based relational database? Version 1.0 is the answer to that question. It was not a from-scratch redesign of the company, and it was not a line-by-line port of every desktop table. The UML and data models were the reference for what FoodHub means. The prototype implements the commercial core that a browser can prove quickly: identity, menu, cart, order, and an administrator who can run that core.

Why a web prototype, rather than another desktop build

- A browser client is easier to put in front of a customer without an installer.
- The desktop design can stay the authority for riders and vehicles while a second channel is tested.
- C++ on the server was a deliberate skill and architecture experiment, not the path of least resistance. The usual web choice would have been another managed runtime. EasySoft chose Crow, CMake, and SQLite instead.
- A completed prototype creates a stable point. Later work can harden this channel or fold it back toward the fuller Windows model. It does not have to do both in one pass.

Problem the prototype addresses

The desktop concept is rich and, in its raw drawings, uneven: composite attributes, repeating groups, derived ages, and conflicting foreign keys appear across diagram versions. A developer who tried to implement every box at once would have encoded those conflicts. Version 1.0 avoids that trap. It implements a coherent subset with explicit rules — stock cannot go negative, a completed order cannot change status, an administrator cannot disable their own account, an image must match its declared type — and it leaves the operational design on paper until those rules are reconciled.

Problem the prototype does not address

It does not dispatch food. It does not pay a rider. It does not know a vehicle. Checkout copy in the interface says that payment stays unpaid until it is confirmed separately, and the database column `payment_status` defaults to Unpaid. There is no delivery fee. There is no map and no dispatch location, even though the future order entity has one. Those omissions are scope, not accidental bugs. They are listed again in Section 4 so a reader of the repository cannot mistake a green “completed” badge for a finished logistics platform.

SECTION 4

Objectives and scope

Objectives that version 1.0 meets

1. Deliver a working web food-ordering application under the EasySoft brand.
2. Keep the concept of the C# .NET FoodHub system recognizable: customers, items, orders, money, and an administrator.
3. Prove a C++ HTTP backend with SQLite and OpenSSL, built by CMake and vcpkg.
4. Produce an interface that a customer can use without training: menu, cart, orders, profile.
5. Leave a written and diagrammed path for a stricter, wider version.

In scope for the prototype

Customer

- Register, sign in, sign out
- Browse, search, and filter the menu
- Cart quantities bounded by stock
- Place an order and read its lines
- Notifications, profile, password change

Administrator

- Register only with the workstation admin code
- Create, edit, and deactivate food
- Upload a PNG, JPEG, or WebP image
- Move an order through a kitchen-style status list
- Enable or disable accounts
- See counts and a monthly bar view

Out of scope, reserved for the future design

The following belong to the Windows and admin design set and are excluded from version 1.0: staff as a role distinct from administrator; rider accounts; viewing or requesting ingredients; assigning a rider to an order; assigning a vehicle; meter readings; dispatch time and dispatch location; delivered versus failed delivery with a reason; rider shifts; rider dependents and identity documents; vehicle brand catalogs; multi-branch operation; formal report generation and export; online card payment.

Success criteria used to call 1.0 complete

Criterion	Evidence in the prototype
A customer can register and order	Register, login, cart, and transactional POST /api/orders
An administrator can run the menu and the board	Food CRUD, image upload, status changes, user activation
Data survives a restart	SQLite file data/foodhub.db, WAL mode
Passwords are not stored in clear text	PBKDF2-HMAC-SHA-256 via OpenSSL, random salt
The desktop model is preserved as design, not forced into this build	draw.io set retained and interpreted in this report

> **Currency rule.** Prices are integers in minor units. One rupee is 100 units. The interface formats them as LKR with two decimal places. This avoids binary floating point in the database. A food saved from the price field is rounded to the nearest minor unit before it is sent.

SECTION 5

Development methodology

FoodHub 1.0 was not run as a heavyweight enterprise method, and it should not be described as one. It was a single-developer prototype with a clear split between what was designed by hand and what was produced with AI assistance. That split is part of the project's identity.

Manual system design

Workflow design and diagram design were done manually in diagrams.net, also known as draw.io. The set covers actors and use cases, context and level-1 data flow, four operational procedures, a class model, a conceptual entity-relationship drawing, logical schemas, and a normalization sequence through third normal form. Evan Keal drew these against the FoodHub Windows application and its administrative responsibilities. They are analysis and design artifacts. They were not generated as a side effect of the Vue screens.

Working by hand matters for the next version. A diagram that has been corrected from 1NF to 3NF shows decisions — which repeating groups were pulled out, which keys stopped doing two jobs. An AI-generated diagram would still need that review. Here the review already exists, including the places the sequence is not yet finished. Section 16 is that review.

AI-assisted interface and implementation

The visual interface was an AI-assisted design, then implemented in Vue, HTML, and CSS. Programming of the C++ service and the Vue client was also AI-assisted, inside Visual Studio Code, with Git as the record. AI assistance did not replace ownership. The brand, the scope decision, the choice to keep C++ on the server, the decision to base the concept on the existing desktop model, and the decision to stop at a prototype all sit with EasySoft and with Evan Keal.

Shape of the work

1. **Recover the concept.** Read the desktop model as actors, orders, items, and administration rather than as a pile of screens.
2. **Cut a web slice.** Customer plus administrator. No rider.
3. **Choose a stack that is allowed to be unusual.** Vue on the front, C++ and SQLite behind it, so the prototype teaches a full path and not only a JavaScript path.
4. **Implement the slice in a form that runs on a developer workstation.** One executable, one Vite dev server, one SQLite file.
5. **Close the prototype.** Mark version 1.0 complete, publish the repository, and write this report so the unfinished operational model is explicit.

What the method did not include

There is no automated test project in the repository. There is no continuous-integration workflow. There is no issue tracker history beyond the Git publication itself, which is a single commit, "First Upload," on `main`. There is no separate UX research record; the interface language ("Freshly made, just for you," category chips, a soft green visual system) is a designed prototype surface, not a tested brand system with user studies. Those gaps are acceptable for a completed prototype. They are not acceptable as the permanent method. Section 18 makes tests, migrations, and a split codebase the first work after 1.0, before new actors.

> **Design authority.** If a screen and a diagram disagree, the screen describes version 1.0 and the diagram describes the future Windows and admin system. Neither is allowed to silently overwrite the other.

SECTION 6

Technology stack

The stack is small on purpose. Each layer has one main job, and the jobs meet at HTTP and SQL.

Layer	Choice	Role in FoodHub 1.0
Interface	Vue 3.5, HTML5, CSS3	Single-file client. Composition API. No Vue Router.
UI kit	lucide-vue-next	Icons for navigation, cart, alerts, and forms.
Charts	chart.js is declared	Not imported. The monthly view is a CSS bar chart. Chart.js is available for a later release.
Front-end tool	Vite 6	Dev server on port 5173. Proxies /api and /uploads to the C++ process.
Service	C++17, Crow	HTTP, JSON, routing. Executable name foodhub.
Build	CMake 3.20+, vcpkg	Manifest dependencies: crow, openssl. Visual Studio 18 2026, x64.
Crypto	OpenSSL libcrypto	PBKDF2, random bytes, SHA-256. Linked as OpenSSL::Crypto, not as a TLS server built by this project.
Database	SQLite amalgamation	third_party/sqlite/sqlite3.c compiled into the executable. File data/foodhub.db.
Editor	Visual Studio Code	Recommended extensions: C/C++ tools, CMake Tools, Vue (Volar).
Version control	Git, GitHub	Repository under aimstudios3001. Branch main.

Why these choices fit the prototype

Vue. The client is one reactive surface with two modes, customer and administrator, switched from the signed-in role. Vue's composition style matches that: a set of refs for page, cart, foods, and session, and a template that renders one mode. A router would have been better for shareable URLs. It was not required to prove the order path, and it is queued for the next architecture pass.

C++ and Crow. Crow gives route macros and JSON without hiding the fact that the server is a native process. The entire service, schema, and route table live in one translation unit. That is readable for a prototype and wrong for a long-lived system. The file is the place a future split starts, not a pattern to copy into version 2.

SQLite. A single file is the right database for a workstation prototype. Foreign keys are on. Journal mode is WAL. A busy timeout of five seconds is set. There is no separate database server to install, which matches the runbook. SQLite will not be the end state if FoodHub becomes a multi-user hosted service with riders on phones; it is an honest engine for version 1.0 and a good place to stabilize the schema before any move.

OpenSSL. Password hashing and session-token hashing go through libcrypto so the prototype does not invent cryptography. The iteration count for PBKDF2-HMAC-SHA-256 is 210,000. Salts and tokens come from RAND_bytes. Login comparison uses CRYPTO_memcmp. That is a serious baseline. It is not a complete security program; Section 11 is specific about what is still open.

What was intentionally not adopted

There is no C# web port, even though the predecessor was .NET. There is no ORM. SQL is written as statements next to the routes. There is no Redis, no message queue, and no object storage. Images land in `data/uploads` and are served by a Crow route. There is no component library such as a full design system; the visual system is project CSS. These omissions keep the prototype installable from the runbook. They also define the hardening work: if any of them is added later, it should be added for a named reason, not because a larger stack looks more finished.

SECTION 7

Tools, build chain, and repository

Design and construction tools

Tool	Use on this project
diagrams.net (draw.io)	All workflow, UML, DFD, class, and ER drawings. Manual. Future-system baseline for the Windows app and admin operations.
Visual Studio Code	Editing C++, Vue, CMake, and the runbook. AI-assisted programming happened in this environment.
CMake + Visual Studio 18 2026	Generates the x64 build. Documented generator string: "Visual Studio 18 2026".
vcpkg	Manifest mode. Installs Crow and OpenSSL for x64-windows. VCPKG_ROOT is expected at C:\vcpkg.
Node.js and npm	Front-end install and `npm run dev` / `npm run build`.
Git and GitHub	History and publication. Remote account aimstudios3001.
AI programming assistance	Implementation support and the visual UI direction. Not a substitute for the manual diagrams.

Documented workstation run

The repository runbook is written for Windows command prompts. Backend: set VCPKG_ROOT, set the administrator code in the environment, configure with the vcpkg toolchain file, build Debug, and start `backend\build\Debug\foodhub.exe`. The process listens on port 8080. Front end: from `frontend`, `npm run dev`, which serves Vite on port 5173 and proxies API calls to 8080. First-time setup also documents installing CMake, cloning and bootstrapping vcpkg, and installing the `crow:x64-windows` and `openssl:x64-windows` ports.

This is a developer run, not a deployment design. The executable's working directory matters because the database path is relative (`data/foodhub.db`) and uploads are relative. The runbook starts the executable from the `backend` directory for that reason. A future hosted build needs an explicit data directory, a release configuration, and a process supervisor. None of those are part of 1.0.

Repository layout that matters

Path	What it is
backend/main.cpp	Schema, cryptography helpers, and every HTTP route
backend/CMakeLists.txt	C++17 target foodhub, SQLite amalgamation, Crow, OpenSSL::Crypto
backend/third_party/sqlite	Vendored SQLite sources compiled into the binary
backend/data	Created at startup for the database and uploads
frontend/src/App.vue	All screens and client behavior
frontend/src/style.css	Visual system: type, color, layout
frontend/vite.config.js	Vue plugin, port 5173, proxy for /api and /uploads
vcpkg.json	Manifest name foodhub-web, version string 1.0.0
docs/images	EasySoft logo and mark used by the README and by this report
.vscode/extensions.json	C++, CMake, and Volar recommendations

The published history is intentionally thin: one commit that introduces the prototype. Later fixes should be new commits on `main` or on short branches, not a rewritten history. This report is a companion to that repository. It does not replace the README, which remains the short public description.

SECTION 8

Architecture of version 1.0

The running prototype has three tiers and a very short middle. The browser never opens SQLite. The C++ process never renders HTML. Vite is a development proxy, not a production application server in this design.

> **Browser (Vue, port 5173)** → HTTP JSON under /api, images under /uploads → **Crow process (port 8080)** → prepared statements → **SQLite file**. Session tokens travel as `Authorization: Bearer`. The browser keeps the token in sessionStorage under the key fh_token, and a small user object under fh_user.

Request path

1. The Vue function `api()` calls `fetch` with a JSON body, or with a raw image body and its content type.
2. Vite forwards `/api` and `/uploads` to 127.0.0.1:8080 during development.
3. Crow matches a route. Almost every route takes a process-wide mutex before touching SQLite.
4. Identity, when required, is the bearer token hashed with SHA-256 and looked up in `sessions`, joined to an active user, with an expiry in the future.
5. JSON responses set `Content-Type: application/json` and `Cache-Control: no-store`.

Concurrency and transactions

Crow is started with concurrency 4, but a single mutex wraps database work. Requests are therefore serialized at the data layer. That is a prototype choice: it avoids shared-connection mistakes on one sqlite3 pointer. It also means the extra threads do not provide database parallelism. The important correctness boundary is not the mutex. It is the order transaction, which uses `BEGIN IMMEDIATE`, rejects an empty cart or any line whose stock is short or whose food is inactive, inserts the order and the snapshotted lines, decrements stock, deletes the cart, writes a notification, and commits. Any exception rolls back.

Two clients, one binary

Customer and administrator are roles on the same user table, not two applications. The Vue shell hides the menu, cart, and order history from administrators, and hides the dashboard, food form, order board, and user list from customers. The server repeats those rules. A customer token cannot create food. An administrator token cannot put items in a cart or place an order through the customer route. Interface hiding is not the security boundary. The route checks are.

What this architecture is good at

It is good at being understood in an afternoon, at running on one machine, and at keeping a consistent story between a green interface and a strict server. It is poor at independent scaling, at team ownership of modules, and at testing a route without starting the whole file. The future architecture in Section 18 keeps the three tiers and changes the inside of the C++ and Vue codebases. It does not throw away the channel that 1.0 proved.

SECTION 9 · CUSTOMER

Functional design — customer

Until someone signs in, the client shows a welcome panel and an authentication card. The panel is the AI-assisted visual front: the line “Your next favorite meal starts here,” a short supporting sentence, and a path into the product. After a customer signs in, the same application becomes a storefront.

Accounts

Registration collects full name, username (at least three characters), email, optional phone, and a password of at least eight characters. The form can claim the administrator role, but the server rejects that claim unless the administrator code matches the environment. A normal customer does not need the code. Sign-in accepts username or email plus password. A successful sign-in returns a token and moves the customer to the menu. Sign-out deletes the server session and clears sessionStorage. If `/api/auth/me` returns no row, the client treats the session as dead and signs out.

Menu and cart

The menu loads active foods and the category list. Seeded categories are Rice & Curry, Burgers, Pizza, Noodles, Beverages, Desserts, and Other. The customer can search by name and filter by category. Each card shows an image if one was uploaded, otherwise a neutral placeholder, plus category, description, price, and remaining stock. Adding to the cart increments quantity. The server refuses a quantity above stock, above 100, or against an inactive item. Quantity zero removes the line. The cart screen shows a line total and an order summary. Delivery is explicitly “Not included.” The customer confirms with a browser confirm dialog, which is acceptable for a prototype and should be replaced by an in-page review before any wider release.

Orders, notices, profile

A placed order returns an id. The client opens order history. Each row shows status, payment status, total, and time. “View items” loads the snapshot lines. The customer cannot edit a placed order. Notifications are created for welcome, sign-in, password change, order placement, and later status changes. The customer can mark them read. Profile updates cover full name and phone. Email and username are not edited in 1.0. Password change requires the current password and a new password of at least eight characters.

Customer rules the server enforces

- Only a customer role may write the cart or place an order.
- Order lines store the food name and unit price at purchase time, so a later menu edit does not rewrite history.
- Stock falls only inside the successful order transaction.
- A customer sees only their own orders, unless the caller is an administrator inspecting a specific order’s lines.

> **Relation to the future “Place an Order” flowchart.** The flowchart starts by asking whether the customer is registered, then either loads or captures details, adds items, calculates a total, and saves. Version 1.0 follows that spine. It does not yet let staff enter an order on the customer’s behalf, and it does not write a dispatch location. Those steps stay on the future side of the boundary.

SECTION 9 · ADMINISTRATOR

Functional design — administrator

An administrator who signs in lands on a dashboard rather than the storefront. The navigation is Dashboard, Food Items, Orders, and Users, plus the shared notification and profile actions. This desk is the web prototype's stand-in for "system administrator," and it also absorbs a little of what the drawings call staff work, because nobody else can move an order. It does not absorb rider work.

Dashboard

Four figures are loaded from `/api/admin/analytics`: customers, active foods, orders, and the sum of totals for orders whose status is Completed. Pending count is also computed on the server and is available to a later screen. Below the figures, months that have orders are drawn as bars scaled to the busiest month. The chart is empty until the first order exists. Completed value ignores orders that are not completed, so a large pending board does not look like revenue.

Food items

The administrator adds a name, category, description, price in rupees, and stock. Edit loads the same form. Deactivate is a soft delete: `active` becomes 0, the public menu hides the row, and history can still point at the old food id. Images are optional, square-friendly, and limited to 5 MB. The server checks the content type and the file signature for PNG, JPEG, and WebP. The stored name is random hex plus the extension. The public URL is `/uploads/`. Path segments and `..` are rejected.

Orders

The board lists every order with the customer's name, money, payment status, and a status control. Allowed values are Pending, Confirmed, Preparing, Ready, Completed, and Cancelled. Pending is the default at insert. Completed and Cancelled are final; the server returns a conflict if a later update is attempted. A legal change writes a notification to the customer: "Order #n is now …". This status list is a kitchen and counter workflow. It is not the drawing's Pending / Dispatched / Delivered set. Both are useful. Section 18 recommends keeping a preparation status and a delivery status rather than overloading one column when riders arrive.

Users

The user list shows name, email, username, role, and whether the account is active. The administrator can disable or enable an account. The server refuses a request that would disable the caller. Disabled users fail the active check at sign-in and at session lookup. There is no screen for editing another person's address, NIC, or dependents, because version 1.0 does not store them.

> **What this desk is not.** It does not assign vehicles, open a rider shift, record a meter, manage ingredients, or generate the reports named on the use-case diagram. Those use cases are drawn for the Windows administrative system. They are specified later in this report and left unimplemented here.

SECTION 10

Implemented data model

The physical schema is created at process start with `CREATE TABLE IF NOT EXISTS`. There is no migration history. That is acceptable only while one developer owns one file. The tables below are the entire persistent model of version 1.0.

Table	Purpose and notable rules
users	full_name, unique username, unique email, phone, salt, password_hash, role checked to admin or customer, active flag, created_at
sessions	token_hash primary key, user id, unix expiry. The raw token is never stored.
categories	Unique name. Seven names are inserted with INSERT OR IGNORE at startup.
foods	Name, category, description, price_minor ≥ 0, stock ≥ 0, image path, active flag
carts	Composite primary key (user, food), quantity > 0
orders	User, status default Pending, payment_status default Unpaid, total_minor, notes, created_at
order_items	Order, optional food id, food name snapshot, quantity, unit price snapshot
notifications	User, title, message, is_read, created_at. The client reads the latest 100.
audit_logs	actor_id, action, created_at. The table is created. No route writes it.

Integrity choices already made

- Foreign keys are enforced. Orders belong to users. Lines belong to orders. Cart rows belong to users and foods.
- Indexes exist on orders by user and notifications by user.
- Food deactivation does not delete order history. The line keeps its own name and price.
- Age is not stored, because the prototype does not store dates of birth. That accidentally matches the good rule in the drawings: age is derived.
- There is no ingredient, rider, vehicle, assignment, dependent, branch, or payment-method table.

Schema pressure the prototype already feels

Status is a free string checked in application code, not a foreign key. That matches speed of development and will drift once delivery statuses are added. Payment status is a column with no payment rows behind it. Notes exist on the order and are always inserted as an empty string; the customer has no notes field in the UI. Phone is a single optional column, which the future model correctly treats as multi-valued. The unused `audit_logs` table is a promise. The next version should either write it on every sensitive change or remove it until it is real. An empty audit table in a completed prototype is documentation of intent, not an audit capability.

> **Physical versus logical.** Do not “upgrade” this schema by executing the 1NF drawing as CREATE statements. The 1NF drawing still shows repeating groups and derived attributes as callouts. The physical target, after the corrections in Section 16, is the 3NF model plus the extra tables those callouts still require.

SECTION 11

Security and sessions

Version 1.0 takes identity seriously for a prototype and leaves the surrounding operational security unfinished. This section states both, because a portfolio reader and a future implementer need the difference.

Controls that are in the code

Control	Implementation
Password storage	Random 16-byte salt, hex encoded. PBKDF2-HMAC-SHA-256, 210,000 iterations, 32-byte output. OpenSSL EVP.
Login compare	CRYPTO_memcmp against the stored hash. Failure is a generic “Invalid credentials,” including an unknown user.
Session	32 random bytes returned to the client. Only SHA-256(token) is stored. Lifetime is 24 hours from issue. Logout deletes that hash.
Administrator creation	Role admin is ignored unless FOODHUB_ADMIN_CODE is set and the submitted code matches it. Otherwise the role becomes customer, except the explicit admin path which returns 403.
Authorization	Each admin route checks the role. Cart and checkout require customer. Order line reads allow the owner or an administrator.
Self-lockout	An administrator cannot set their own active flag to 0.
Order finality	Completed and Cancelled cannot be moved.
Uploads	Administrator only, 5 MB, signature check, random filename, rejection of path traversal, nosniff on the image response.
SQL	Values are bound on prepared statements. Schema SQL is constant.

Controls that are not in the code

- No transport security is configured by the application. The runbook is local HTTP. OpenSSL is used as libcrypto. Crow is not started as a TLS listener in this project.
- The bearer token sits in sessionStorage. Any script running in the origin can read it. A future release should prefer an HttpOnly, Secure, SameSite cookie, or accept the bearer model only with a strict content security policy and no injected script.
- There is no rate limit, no lockout, and no delay on failed sign-in.
- Password change compares hashes with ordinary string inequality, not CRYPTO_memcmp. Login does the constant-time compare. The change path should match it.
- The runbook shows a sample administrator code for a private workstation. That sample is not a production secret and must not be reused on any shared or hosted system. This report deliberately does not repeat the value.
- Uploads are unauthenticated to read if the random name is known. Names are long, but they are not an access-control policy.
- There is no backup, no encryption at rest, and no audit trail, despite the unused audit table.
- CORS is not a designed policy; the Vite proxy makes the browser regard the API as same-origin in development.

> **Deployment rule for any demo beyond the author’s machine.** Do not expose the Debug executable and the sample administrator code on a network. Version 1.0 is a completed prototype, not a public production deployment. Hosting it is a later project with TLS, a replaced secret, and the session changes above.

SECTION 12 · INTERFACE

Service interface

The prototype has no separate API document. The route table in `main.cpp` is the contract. Methods not listed are not implemented. JSON is the body format except for image upload, which is the raw file with a content type.

Method and path	Who	Behavior
GET /api/health	Public	Returns status ok
POST /api/auth/register	Public	Creates a user. Admin only with the environment code.
POST /api/auth/login	Public	Issues a bearer token for 24 hours
GET /api/auth/me	Signed in	Profile row for the token
POST /api/auth/logout	Bearer	Deletes the session hash
PUT /api/profile	Signed in	Updates full name and phone
POST /api/auth/password	Signed in	Changes password if the current one matches
GET /api/categories	Public	Id and name, ordered by name
GET /api/foods	Public	Active foods. Optional q= name filter.
POST, PUT, DELETE /api/admin/foods	Admin	Create, update, soft-delete
POST /api/admin/foods/:id/image	Admin	Stores a checked image and saves its path
GET /uploads/:name	Public	Returns one upload file or 404
GET, PUT /api/cart and /api/cart/:id	Customer for writes	Read lines; set quantity, including delete at 0
POST, GET /api/orders	Customer to create; owner to list	Transactional checkout; own history
GET /api/orders/:id	Owner or admin	Line items for one order
GET /api/notifications	Signed in	Latest 100 notices
POST /api/notifications/read	Signed in	Marks that user's notices read
GET /api/admin/orders	Admin	All orders with customer name
PUT /api/admin/orders/:id/status	Admin	Moves status if it is not final
GET /api/admin/users	Admin	Account list
PUT /api/admin/users/:id/active	Admin	Enable or disable, not self-disable
GET /api/admin/analytics	Admin	Counts, completed value, monthly buckets

Error bodies are JSON with an `error` string. The client surfaces that string in a dismissible alert. Validation messages are plain language ("Check name, username, email and password", "Food unavailable or insufficient stock"). There is no error-code catalog. A later public API would need stable codes; this prototype does not pretend to have them.

SECTION 12 · INTERFACE

User interface and visual system

The interface was produced as an AI-assisted design and implemented directly in project CSS, not as a separate design-tool file that the code then approximated. The system diagrams stayed in draw.io. The screen design did not. That is a conscious split: structure was authored by hand, surface was generated with assistance and then owned in Vue.

Visual language

The palette is a fresh-food green on a near-white ground, with a small coral accent. It is the same language this report uses, so the document and the product are recognizably one project.

Token	Value	Use
Ink	#1d3128	Text
Leaf	#169e55, #19a45b, #1b9a56	Brand mark, primary buttons, emphasis
Leaf wash	#eaf7ee, #f0f9ed to #e4f4e4	Selected navigation, hero
Ground	#fbfdf9	Page background
Coral	#ed7352	Unread notification dot
Type	Outfit for names and titles, DM Sans for UI text	Loaded in the style sheet

Screen structure

A sticky white bar carries the wordmark “foodhub,” the tag “Freshly made, just for you,” role-specific navigation, and icon buttons for notices, profile, and sign-out. The customer home is a two-column hero and then a menu grid. Cards are image-led. The cart is a list plus a summary panel. Administrator pages reuse the same panels, forms, and pills so the desk does not look like a second product. The footer is a single quiet line.

Interaction limits worth fixing later

- Navigation is a page name in memory. Refreshing deep inside the desk returns to the data, but there is no URL for “orders” or “food item 12.”
- Order lines open in a browser alert. That is not a receivable layout, and it is poor on a phone.
- Confirmations for checkout, deactivate, and similar actions use the browser confirm dialog.
- The layout is designed around a wide shell (max width 1200px) with a two-column hero. A small phone was not the primary 1.0 target.
- Chart.js is a dependency, but the dashboard does not use it, so the visual chart has no text alternative beyond the counts printed under the bars.

None of these block the claim that 1.0 is a completed prototype. They do block a claim of a polished consumer product. The future UI work is to keep this green system, introduce real routes and dialogs, and add new desks — staff, rider, reports — without inventing a second visual language.

SECTION 12 · DELIVERY

How version 1.0 was delivered

Build identity

Artifact	Identity
Front-end package	foodhub-frontend 1.0.0, type module
vcpkg manifest	foodhub-web 1.0.0, dependencies crow and openssl
C++ standard	17
Executable	foodhub, Debug build documented for the workstation
Source publication	Commit “First Upload” on main
License stance in source headers	Copyright 2026 EasySoft. All rights reserved.

Runtime shape

Two processes, both local. The C++ process creates `data/uploads`, opens or creates the database, applies the schema, seeds categories, and serves HTTP until it is stopped. The Vite process serves the interface and proxies. A production-style front-end build exists as `vite build`, but the repository does not include a static-file host that also reverse-proxies the API. Shipping a built front end next to the executable is future work. Until then, “run the system” means the two-step runbook.

Quality bar actually achieved

The quality bar for 1.0 is functional completeness of the in-scope slice, readable source, and a design set that outlives the slice. It is not measured coverage, not a penetration test, and not a performance budget. No latency numbers are claimed in this report because none were recorded as a formal test. The order path is transactional. The password path uses a modern password hash. Those are engineering properties, verified by reading the implementation, not by a separate test report. EasySoft should not invent a test report after the fact. The honest statement is: version 1.0 was completed without an automated test suite, and the next version starts by adding one around stock, authorization, and status finality.

Handover contents

- The GitHub repository named in the control sheet.
- The EasySoft README inside that repository, which is the short public statement of the same project.
- The draw.io figure set reproduced from Section 13 onward.
- This report, document ES-FH-SDR-1.0.
- The developer portfolio at evan-keal-portfolio.vercel.app, where the project sits in the wider EasySoft body of work.

> **Close of the prototype half of this report.** Everything above is version 1.0 as built. Everything below is the system FoodHub is designed to become, drawn for the Windows application and for administration, and not yet built in the web prototype.

SECTION 13

Design boundary and operating model

> **Scope stamp for every figure that follows.** These drawings were prepared manually in diagrams.net for the FoodHub Windows application and for its administrative side. They are the baseline for future development of the FoodHub system. They include actors and data that version 1.0 does not implement. A feature appears in a figure because it is planned, not because the web prototype already does it.

Actors the future system recognizes

Actor	Responsibility in the drawings	In web 1.0?
Customer	Places an order, can see ingredients and order status	Partial. Orders yes. Ingredients no.
Staff member	Enters customer and order detail, views ingredients, updates order status, records dispatch time	No separate role. Admin updates status only.
Rider	Sees assigned orders, updates delivery, records meter readings	Not implemented.
System administrator	Customers, items and ingredients, vehicles, riders, dependents, reports, vehicle assignment	Partial. Users, foods, and a simple dashboard only.

Operating picture the drawings describe

A customer order is not finished when it is saved. Staff confirm the detail, the kitchen state changes, a rider and a motorbike are chosen, a start meter is written, dispatch time is stored, and the status becomes dispatched. The rider then either records delivery or records failure with a reason. At the end of a shift the rider records the end meter, releases the vehicle, and the records are saved. In parallel, administration maintains the master data that makes this possible: people, identity and contact details, who depends on a rider, which vehicles exist, and which reports the business needs.

One conceptual drawing also shows a branch: a place with a name, address, and contact, related to items and to riders who work there. Later logical models drop the branch. This report treats branch as undecided. It should not enter the first physical schema unless EasySoft decides that FoodHub is multi-site. A single-site system can add a branch key later more easily than it can remove a half-used one.

How the web channel should absorb this model

The future system is not required to abandon the browser. The Windows design is the operational authority: roles, vehicles, meters, dependents, reports. The web prototype is the channel authority for the customer menu and the start of the admin desk. A sound future product is one system with more than one client. Staff may keep a desk application. Riders may need a narrow mobile web or a small native client. Customers can stay on the Vue storefront. They should share one database design, the corrected 3NF model, rather than two drifting schemas.

The pages that follow reproduce each drawing and state what a developer must take from it. They are part of the specification for versions after 1.0.

SECTION 14 · FIGURE 1

Use case diagram

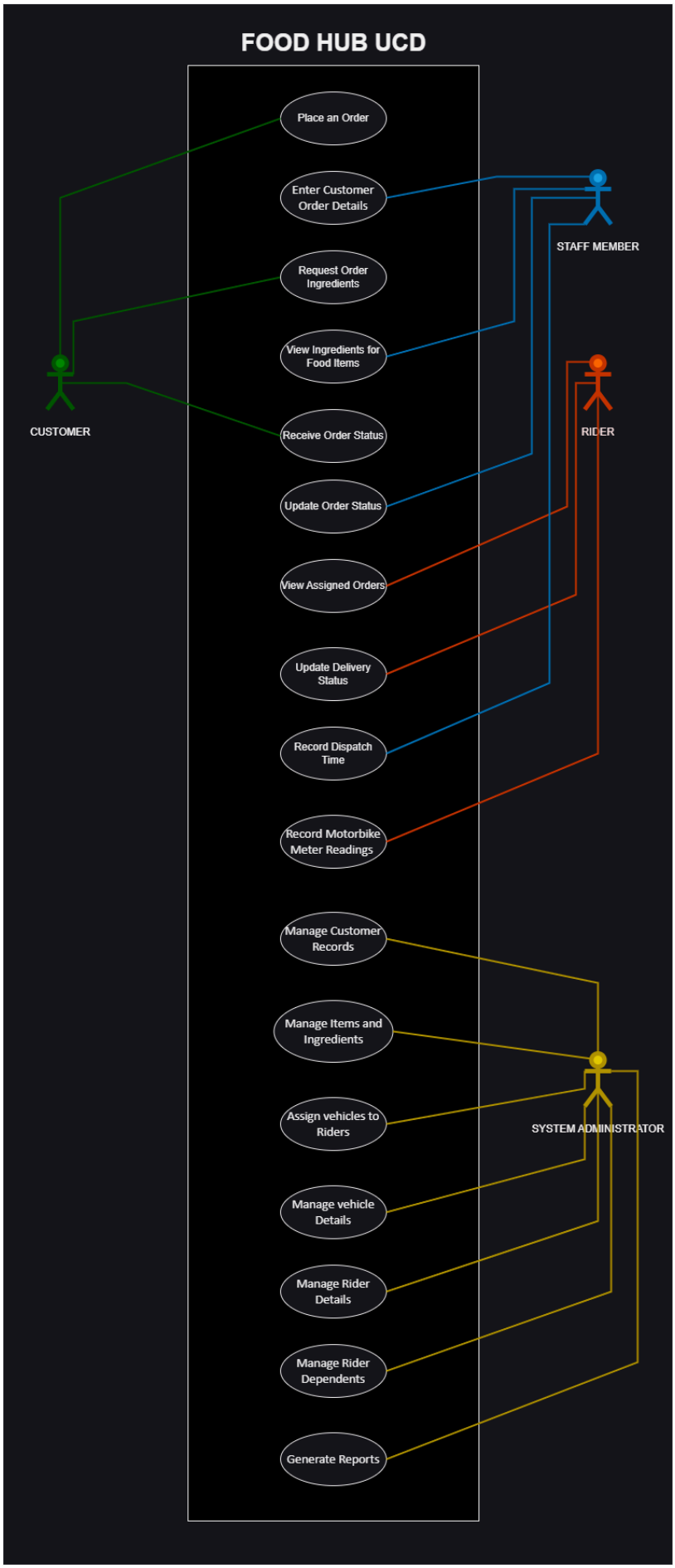


Figure 1. Food Hub use case diagram, drawn in diagrams.net for the Windows application and administrative operations. Not a map of web prototype screens.

SECTION 14

Reading the use cases

The diagram is a classic use-case layout. The system boundary holds the ovals. Actors stand outside it and associate to the cases they perform. There is no include or extend relationship drawn, so each oval is an independently stated goal. That is enough for a prototype's future backlog. It is not yet a fully factored use-case model; common steps such as "identify the order" will want include-relationships when the cases are written out in full.

Customer goals

Place an order. The primary success path is the flowchart in Figure 4. **Request order ingredients** and **view ingredients for food items** are transparency goals. They require the item–ingredient relationship that version 1.0 does not have. **Receive order status** is partly met today by history, pills, and notifications, but the statuses the customer will eventually receive include dispatch and delivery, which the prototype cannot send.

Staff goals

Staff enter customer details and order details, view ingredients, update order status, and record dispatch time. Staff are the counter and the kitchen desk. They are not the system administrator. In a future build, staff should not manage rider dependents or vehicle master data. The prototype collapses staff and administrator into one login because the slice was small. The use-case diagram says that collapse should not survive.

Rider goals

The rider views assigned orders, updates delivery status, and records motorbike meter readings. The rider does not edit the menu and does not see dependents in this diagram. A rider client can therefore be narrow: today's assignments, one status update, two meter fields, and a failure reason. That narrowness is a design gift. It should be preserved.

Administrator goals

The administrator manages customer records, items and ingredients, vehicle assignment, vehicle details, rider details, rider dependents, and reports. This is master data and oversight, not minute-by-minute dispatch. Dispatch time sits with staff. Meter readings sit with the rider. The administrator assigns the vehicle and can inspect the result. Reports are the administrator's read model across all of the above.

> **Backlog rule.** Each oval is a future capability with an owner. Version 1.0 may cover only the center of "place an order," "receive order status," "update order status," and a thin version of "manage customer records" and "manage items." Every other oval is unstarted.

SECTION 14 · FIGURE 2

Context diagram — DFD level 0

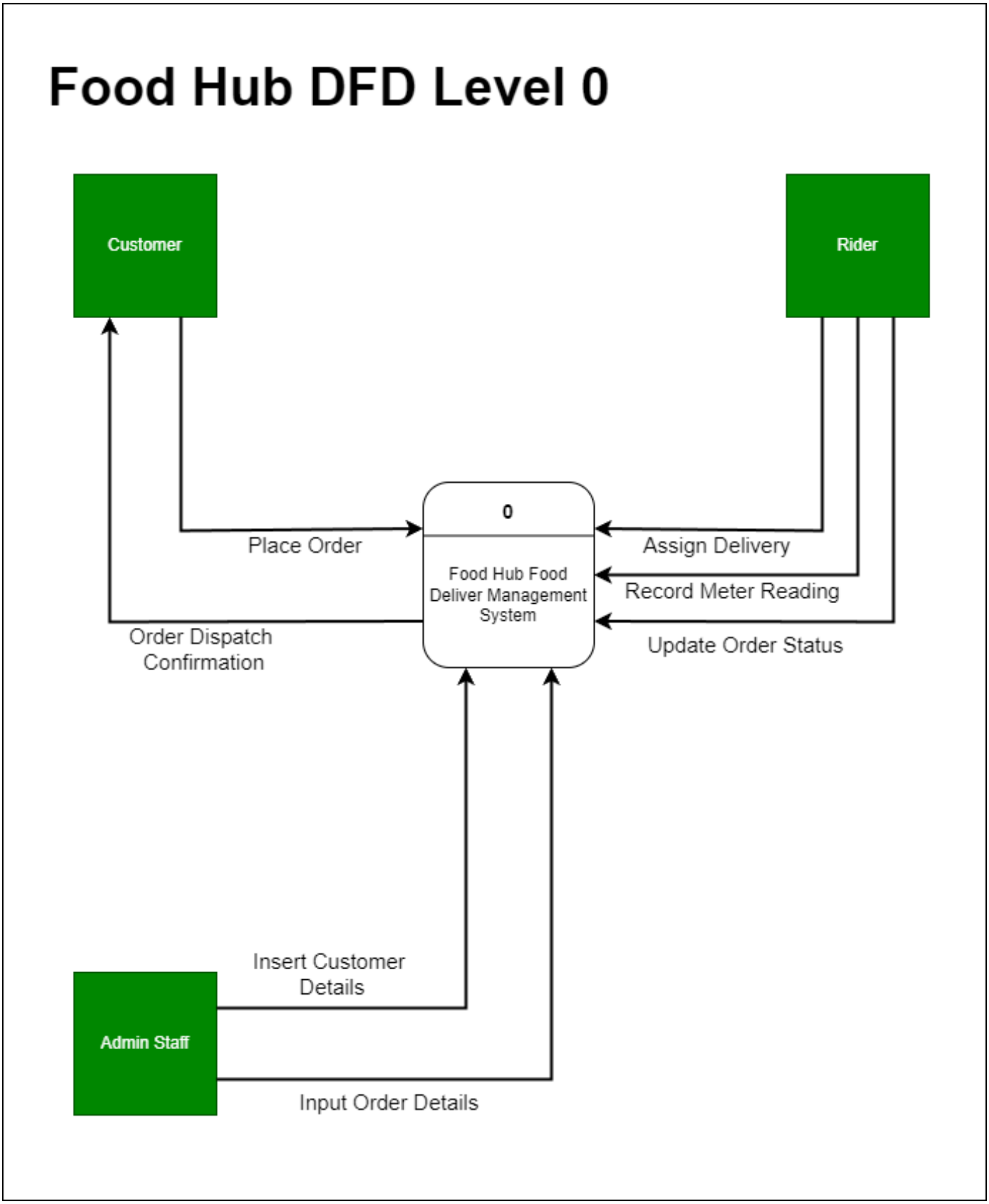


Figure 2. Context diagram. One process, "Food Hub Food Deliver Management System," and three external parties. Windows and admin design baseline, not the web prototype's process model.

SECTION 14

What the context diagram fixes

Level 0 is deliberately blunt. The whole future system is one process. Three external parties exchange flows with it. Customers place an order and receive order-dispatch confirmation. Admin staff insert customer details and input order details. Riders assign a delivery, record a meter reading, and update order status. No data store is visible, because a context diagram does not open the system.

The wording on the drawing says “Food Deliver Management System.” That name is more honest about the future product than “menu prototype.” Delivery is in the name. Version 1.0 should keep the FoodHub name and should not adopt this process name in code until a delivery flow exists. Using the longer name now would advertise a capability the executable does not have.

Flows that must exist as data, not only as arrows

- **Place order.** Customer identity, lines, quantities, prices, and, in the future model, dispatch location and payment method.
- **Order dispatch confirmation.** A message back to the customer when a rider and a time have been recorded. The prototype’s notification table is the seed of this flow and currently fires on kitchen status, not on dispatch.
- **Insert customer details / input order details.** Staff-originated writes. The web prototype lets the customer insert their own account and their own order. A staff path is additional, not a replacement, unless EasySoft drops self-service.
- **Assign delivery, record meter, update order status.** Rider-originated writes. They need rider authentication. They must not be open administrator forms that a rider cannot reach on the road.

The context diagram omits the system administrator as a fourth external party, even though the use-case diagram includes one. Master-data maintenance is treated as inside the staff/admin operational world rather than as a top-level context flow. That is a simplification. When reports and dependent records are built, the administrator should appear on a revised context diagram so oversight is not hidden inside “admin staff.” Until that revision, implementers should follow the use-case diagram for actor boundaries and the context diagram for the main delivery conversation.

SECTION 14 · FIGURE 3

DFD level 1

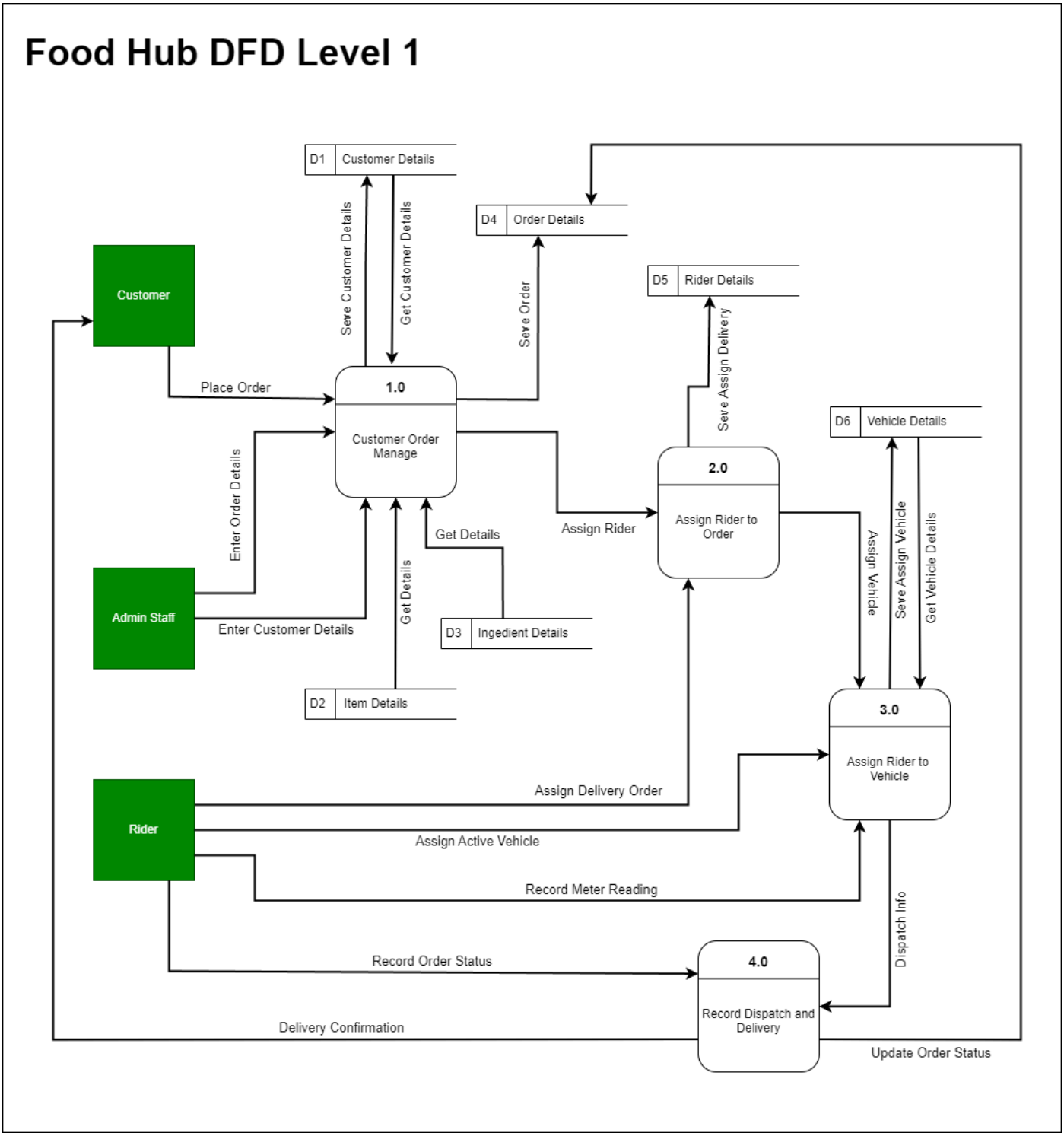


Figure 3. Level 1 processes and stores D1–D6. Labels "Seve" on the drawing mean Save. Windows and admin design baseline.

SECTION 14

Level 1 processes and stores

Level 1 opens the single context process into four processes and six stores. This is the best single picture of how the future Windows and admin system moves information. It is also where version 1.0 can be compared without wishful reading.

ID	Name on the drawing	Future responsibility	Web 1.0
1.0	Customer Order Manage	Place and enter orders, read and save customers, items, ingredients, and orders	Partial. No ingredient store. No staff entry.
2.0	Assign Rider to Order	Choose a rider for an order and record the assignment	Absent
3.0	Assign Rider to Vehicle	Bind a rider to a vehicle for the work, using vehicle details	Absent
4.0	Record Dispatch and Delivery	Dispatch information, meter context, delivery confirmation, status back to the customer	Absent. Admin status is not this process.

Stores

Store	Drawing name	Future contents	Closest 1.0 table
D1	Customer Details	Identity, address, contacts	users, and only the thin account columns
D2	Item Details	Sellable food	foods, categories
D3	Ingredient Details	Recipe lines	None
D4	Order Details	Header, lines, status, dispatch	orders, order_items. No dispatch columns.
D5	Rider Details	Rider master and assignment history	None
D6	Vehicle Details	Vehicle master and assignment	None

The diagram's save labels are misspelled "Seve." Implementation names should be ordinary English. More important than the spelling is direction. Process 1.0 both reads and writes D1, D2, D3, and D4. Process 2.0 writes rider assignment into D5 and reads toward process 3.0. Process 3.0 reads and writes D6. Process 4.0 sits at the bottom of the page and is the only process that owes the customer a delivery confirmation. A future module map can follow these four processes almost directly: ordering, rider assignment, vehicle assignment, dispatch and delivery. Master-data maintenance from the use-case diagram sits across D1–D6 and can be a fifth module, administration, even though level 1 does not draw it as its own bubble.

> **Dependency rule taken from this diagram.** Do not build process 4.0 before 2.0 and 3.0 have somewhere to write. Do not build 2.0 before D5 exists. Do not pretend D3 exists because foods have a description field. A description is not an ingredient store.

SECTION 14 · FIGURES 4 AND 5

Workflows — order and dispatch

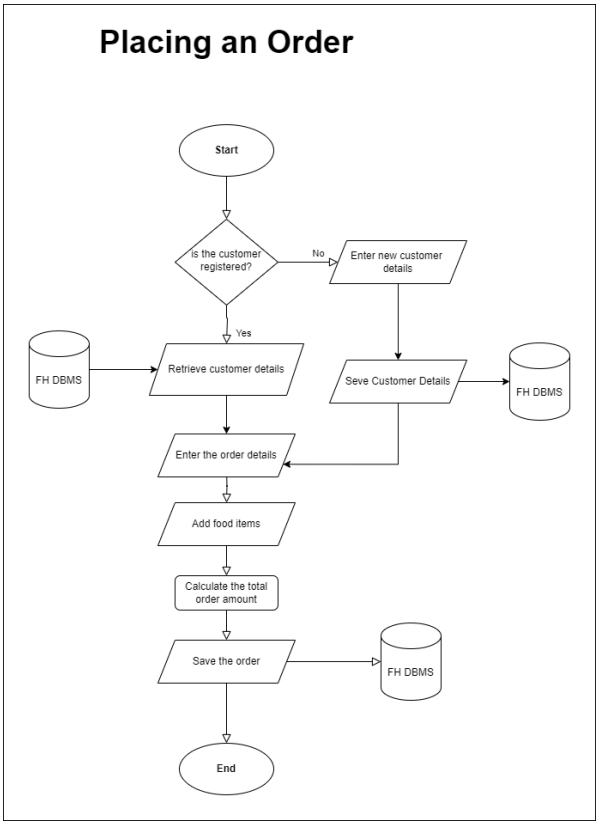


Figure 4. Placing an order. “Seve Customer Details” means save. The database is labeled FH DBMS.

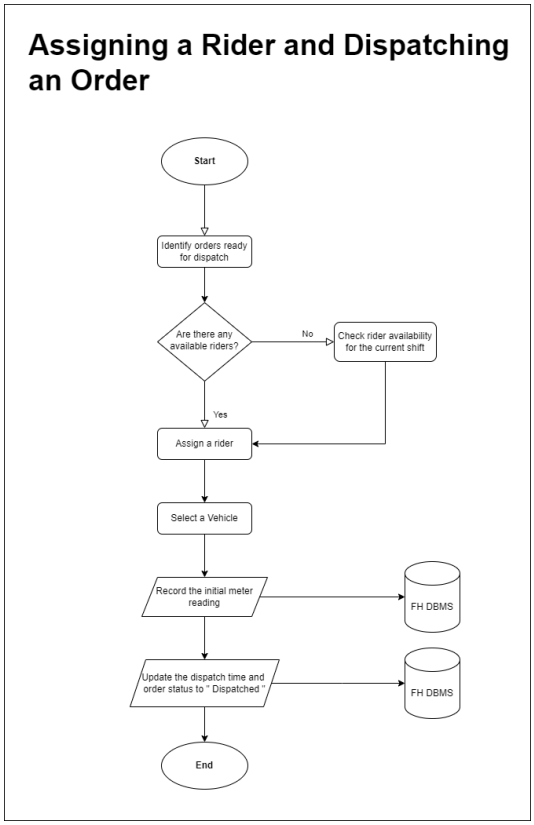


Figure 5. Assign a rider, select a vehicle, record the start meter, set status to Dispatched.

Figure 4, as a future procedure

Start. If the customer is not registered, staff or the customer enter details and the system saves them. If the customer is registered, details are retrieved. Then order details are entered, food items are added, the total is calculated, and the order is saved. Version 1.0 automates the registered path for a signed-in customer and calculates the total in SQL from cart quantity times price. The unregistered path is self-service registration, not a staff form. The future staff form should write the same customer store, not a second one.

Figure 5, entirely future

The procedure assumes orders already exist and are ready for dispatch. It asks whether any rider is available. If not, someone checks the current shift and then assignment continues. A vehicle is selected. The initial meter is saved. Dispatch time is saved and the status becomes Dispatched. None of these steps have tables or screens in version 1.0. “Ready for dispatch” should be defined as a preparation status — the prototype’s Ready is a reasonable predecessor — so the kitchen workflow and this dispatch workflow meet at a named state instead of guessing.

SECTION 14 · FIGURES 6 AND 7

Workflows — delivery and the shift

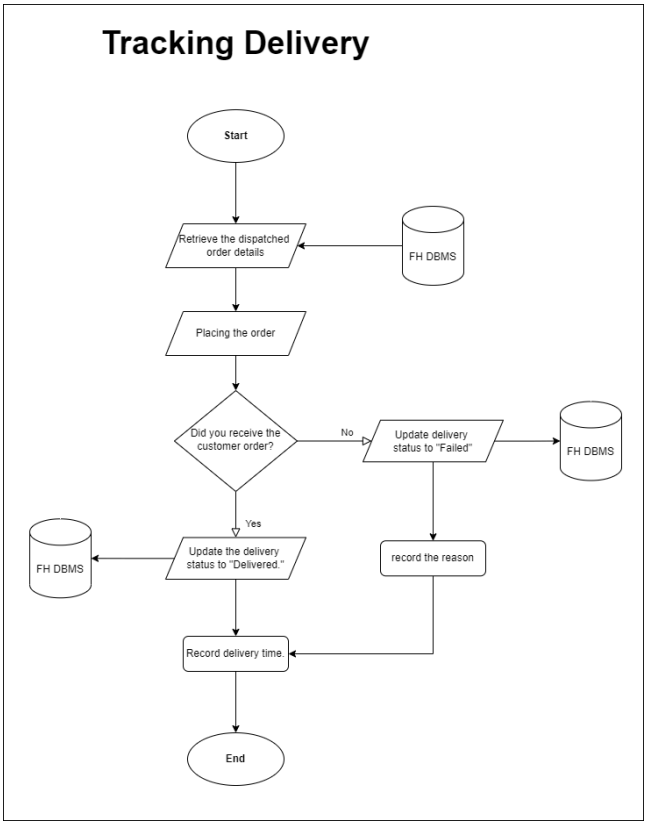


Figure 6. Tracking delivery. Success sets Delivered. Failure sets Failed and records a reason. Both record a time.

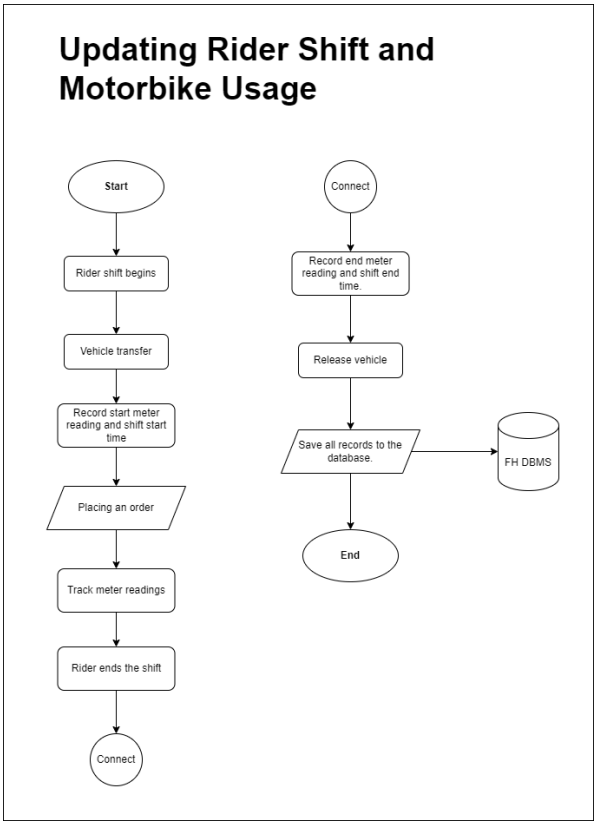


Figure 7. Shift start, vehicle transfer, start meter, work, end meter, release vehicle, save.

Figure 6

The rider retrieves the dispatched order. The diagram's middle step is labeled "Placing the order," which in context means carrying out the drop, not creating a new order. The decision is whether the customer received it. Yes leads to Delivered. No leads to Failed plus a reason. Both paths record a delivery time and end. The prototype has neither status. Adding them later must not destroy Completed as a kitchen concept. Recommended pair: preparation status and delivery status. Failed is a delivery outcome. It should still keep the order financially visible. A failure reason belongs in its own column or in an order-event row, not in a overwritten notes blob.

Figure 7

The shift is a second procedure that shares the meter with Figure 5. At the start: rider begins, vehicle is transferred, start meter and shift start are recorded. During the shift the rider places or carries orders and tracks readings. At the end: end meter and shift end, release the vehicle, save. The right-hand "Connect" terminals on the drawing are continuation markers between the two columns, not a network login. A future assignment row should store shift start, shift end, meter start, and meter end. Distance is end minus start and should be computed, not typed. A vehicle that has been released must be selectable again. A vehicle that has an open assignment must not be silently double-booked. The drawing does not state the double-booking rule; this report adds it as a required business rule before implementation.

SECTION 15 · FIGURE 8

Class diagram

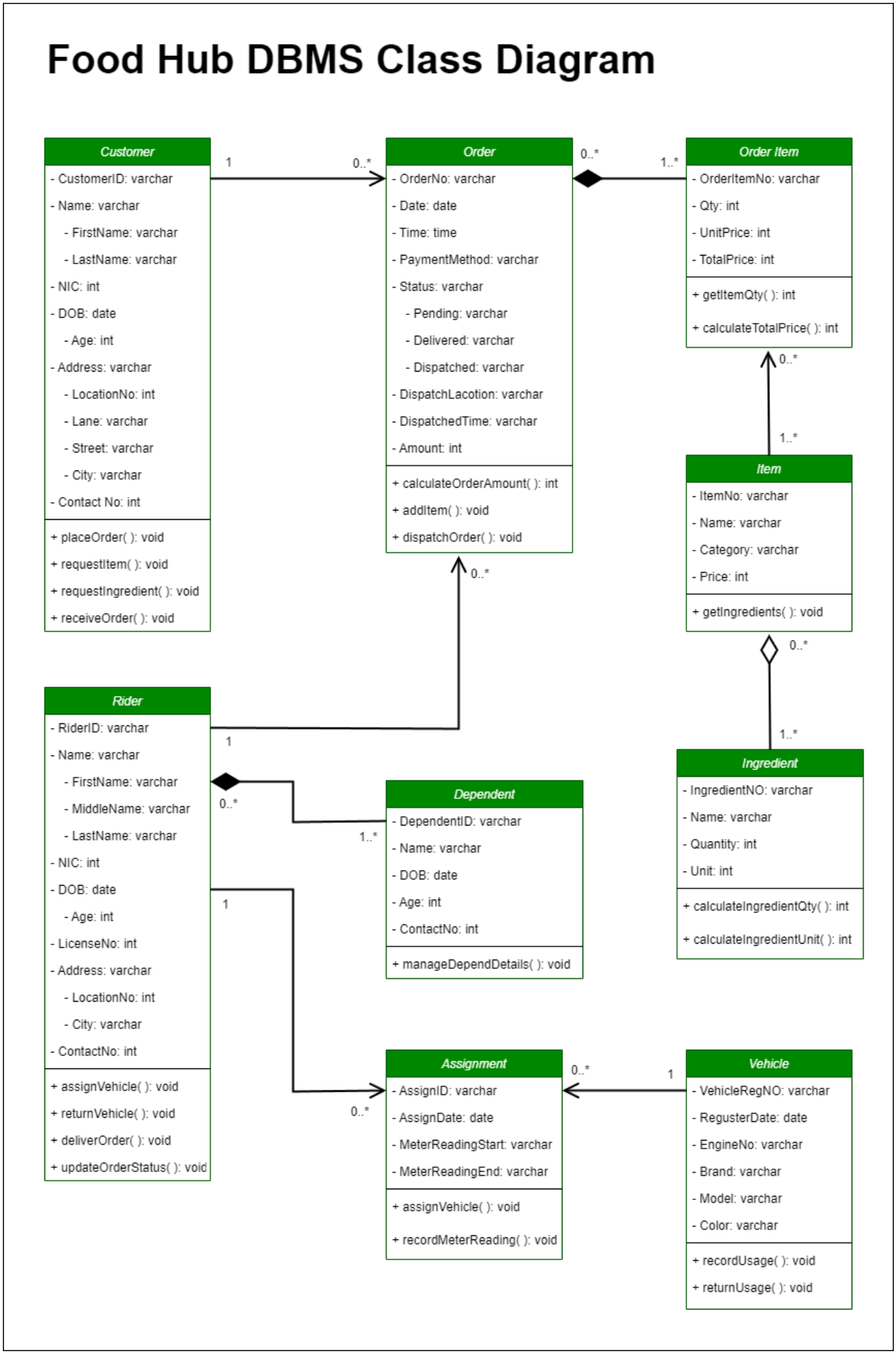


Figure 8. Class diagram for the future FoodHub model. Composition is used for order lines and for rider dependents. Windows and admin design baseline.

SECTION 15

What the classes decide

The class diagram is the object view of the same future system. It is useful because it states operations, not only columns. It is also where a few names must be cleaned before they become code. Dispatch location is labeled DispatchLacotion. Vehicle registration date is labeled RegusterDate. Those are transcription errors. The intended names are dispatch location and registration date.

Relationships worth keeping

- A customer places zero or many orders. An order belongs to one customer. Matches the prototype's users-to-orders foreign key.
- An order is composed of order items. The filled diamond says an order line has no independent life. The prototype agrees: lines are inserted with the order and are not edited as a separate catalog.
- An order item is priced from an item. The class stores quantity, unit price, and total price, and offers a total calculation. The safer object is the one that stores quantity and unit price and calculates the total. Storing the total as well is a cache, not a second fact.
- An item aggregates ingredients. The diamond is aggregation, so an ingredient can outlive a single item. That supports a shared ingredient catalog, which the later many-to-many tables also want.
- A rider has dependents and has assignments. An assignment points at one vehicle. A vehicle records usage. This is the object form of Figures 5 and 7.

Operations that should become services

Customer: place order, request item, request ingredient, receive order. Order: calculate amount, add item, dispatch. Order item: quantity and total. Item: get ingredients. Ingredient: quantity and unit calculations. Rider: assign vehicle, return vehicle, deliver order, update status. Dependent: manage details. Assignment: assign vehicle, record meter. Vehicle: record usage, return usage. Version 1.0 implements only place order, add item, calculate amount, and a narrow update status. The other operations are the method list for the future modules named in the level-1 diagram.

Composition of dependents

The filled diamond from rider to dependent says dependents are destroyed with the rider record. That may be true of a purely private subsystem and too strong for personal data. Before implementation, EasySoft should decide whether deleting a rider deletes dependents or whether the rider is deactivated and the dependent rows are retained or anonymized. The diagram currently chooses cascade. The report flags it as a policy decision, not as a harmless drawing detail.

> **Status fields inside Order.** The class lists Status and also Pending, Delivered, and Dispatched as separate attributes. That repeats one fact. The 3NF drawing later pulls status out. Follow the later drawing, and prefer one status history over three flags that can contradict each other.

SECTION 15 · FIGURE 9

Conceptual entity-relationship diagram

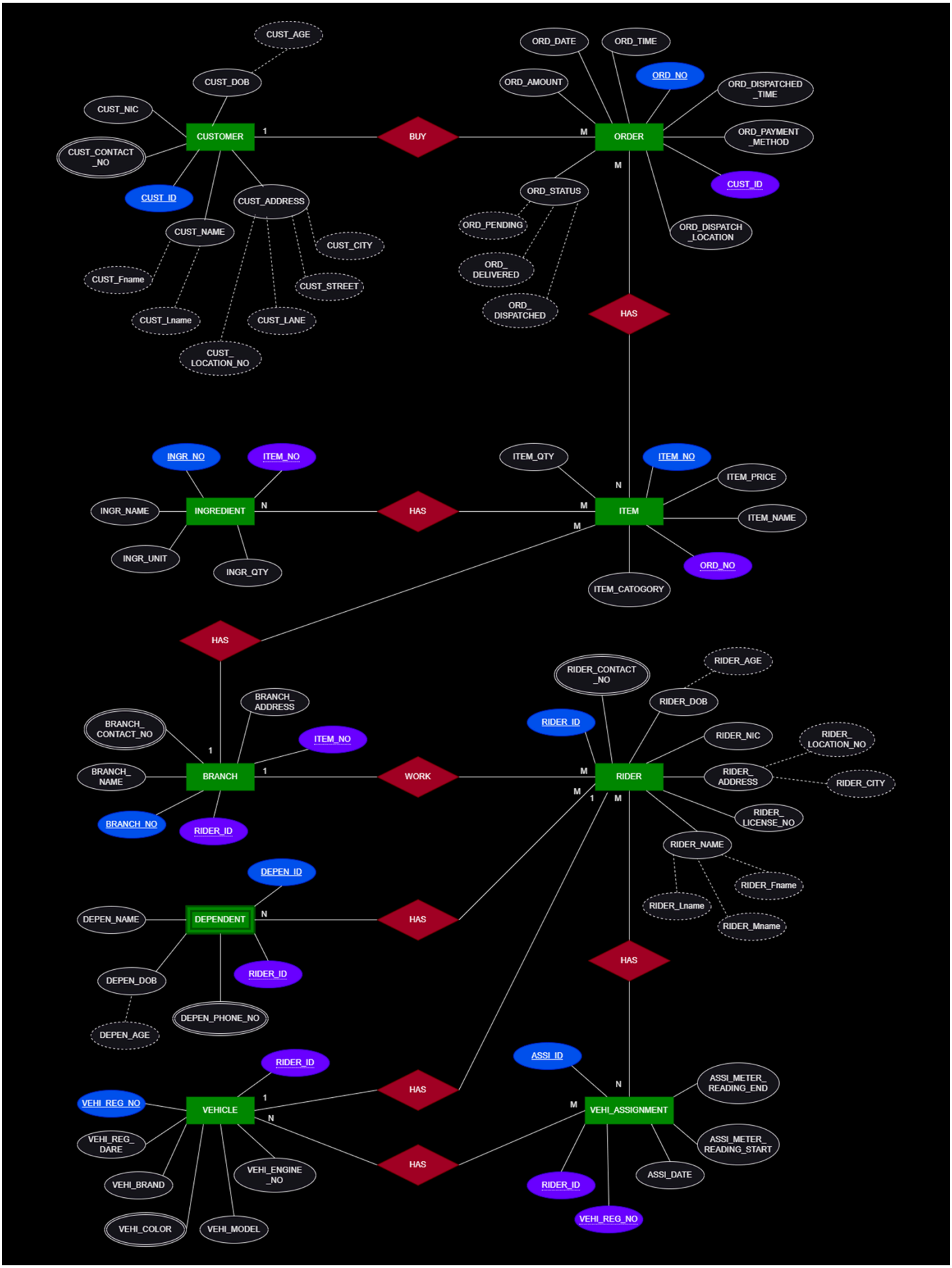


Figure 9. Conceptual ERD. Keys are emphasized on the drawing. Derived attributes use a dashed outline. This is the only figure that centers a BRANCH entity. Windows and admin design baseline. Enlarge the source image for attribute-level reading.

SECTION 15

Entities on the conceptual model

Figure 9 is a Chen-style conceptual model: rectangles for entities, diamonds for relationships, ovals for attributes, underlined or highlighted identifiers, and dashed ovals for derived values. It is the widest picture in the set. Later logical drawings drop and reshape pieces of it. Where they conflict, the conflict is recorded here and resolved in Section 16.

Entity	Identifier	Relationship the drawing states
CUSTOMER	CUST_ID	Buys many ORDERs. Name and address are composite. Age is derived from date of birth. Contact number is multi-valued.
ORDER	ORD_NO	Belongs to a customer. Has many items. Carries date, time, amount, payment method, dispatch location, dispatch time, and a status cluster (pending, delivered, dispatched).
ITEM	ITEM_NO	Has ingredients and appears on orders. Name, category, price, quantity.
INGREDIENT	INGR_NO	Many-to-many with items in this drawing. Name, unit, quantity.
BRANCH	BRANCH_NO	Has items. Riders work at a branch. Name, address, contact. Not repeated in the later logical models.
RIDER	RIDER_ID	Works at a branch, has dependents, has vehicles and assignments. Composite name and address. Age derived. Contact multi-valued.
DEPENDENT	DEPEN_ID	Belongs to a rider. Name, date of birth, derived age, multi-valued phone.
VEHICLE	VEHI_REG_NO	Registration date, engine, brand, model, multi-valued color. Related to riders and to assignments.
VEHI_ASSIGNMENT	ASSI_ID	Date, meter start, meter end. Links rider and vehicle.

Derived attributes

CUST_AGE, RIDER_AGE, and DEPEN_AGE are derived from dates of birth. They must not become stored columns. TOTAL price on an order line, shown as derived on later logical drawings, is the same kind of attribute: compute it. ORD_AMOUNT on the order is a stored total in the prototype today, calculated once at checkout from the lines. That is a reasonable snapshot of what was charged. It should be recomputed if lines ever become editable. In the future model, if lines are immutable after save, a stored order amount is a cache and should equal the sum of line totals.

Branch, held back

BRANCH is the largest conceptual idea that the normalization sequence does not carry forward. Possible readings: the author explored a multi-branch FoodHub and then scoped the logical model back to one operation; or the branch was left for a later diagram that was not drawn. This report chooses the second as a risk, not as a silent deletion. If the business is one kitchen, omit BRANCH. If the business is several kitchens, the logical model is incomplete and needs a new pass before coding: items may be global or branch-specific, riders work at a branch, and orders are fulfilled by a branch. Version 1.1 should not guess.

SECTION 16 · FIGURE 10

Early logical attributes

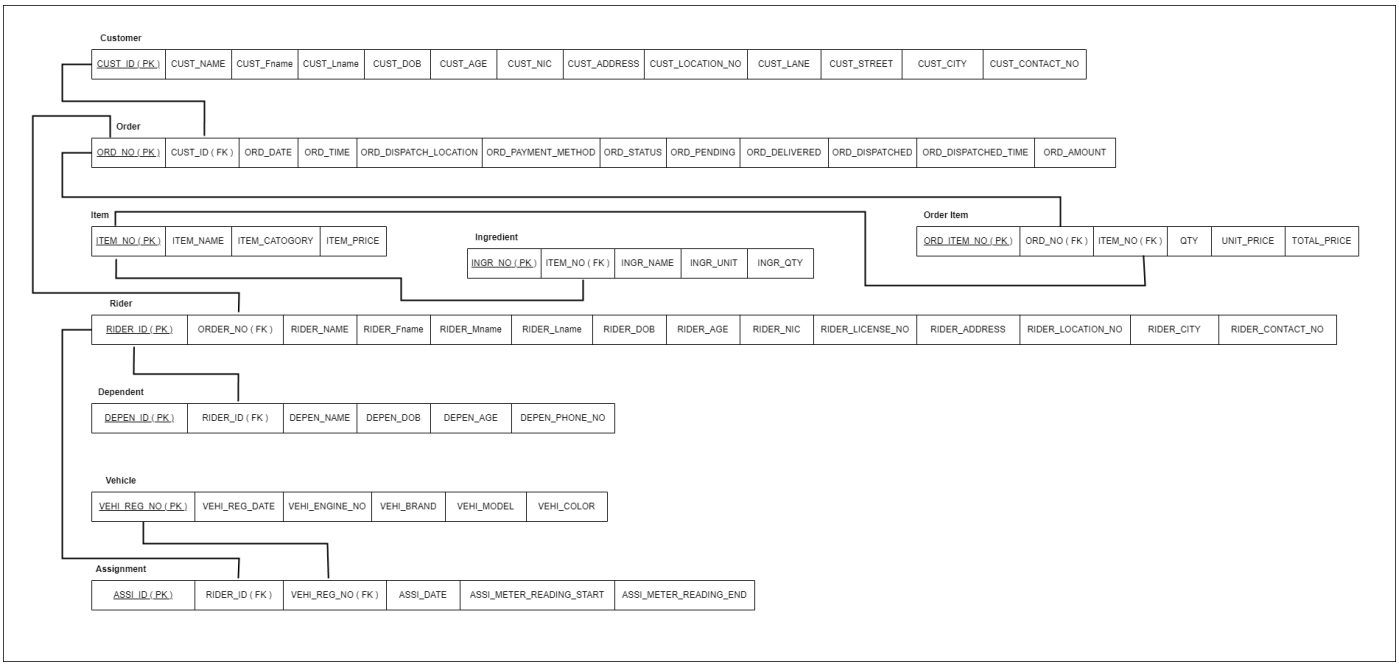


Figure 10. Attribute-level logical sketch. Primary and foreign keys are marked on the drawing. Several columns shown here are later removed or relocated. Windows and admin design baseline.

Figure 10 is a single-page attribute list: Customer, Order, Item, Ingredient, Order Item, Rider, Dependent, Vehicle, and Assignment. It is the bridge from the conceptual ovals to tables. It is not yet normalized. Customer still carries a full name and the pieces of a name, a full address and the pieces of an address, a stored age, and one contact number. Order still carries status and the three status flags together. Rider carries an ORDER_NO foreign key, which would mean a rider row points at one order and cannot be the master record of a person who delivers many orders. Vehicle color is a single column here even though the conceptual model called color multi-valued. Ingredient carries ITEM_NO, which chains each ingredient to one item.

This figure is retained because it shows the starting facts the author did not want to lose: registration numbers, license numbers, NIC, engine numbers, meter readings, dispatch location, payment method. Normalization is allowed to move those facts. It is not allowed to forget them. The next figure boxes the same idea with relationship lines, and it still contains the rider-to-one-order foreign key. That key is the defect the 2NF model later repairs.

SECTION 16 · FIGURE 11

Logical schema before the normal forms

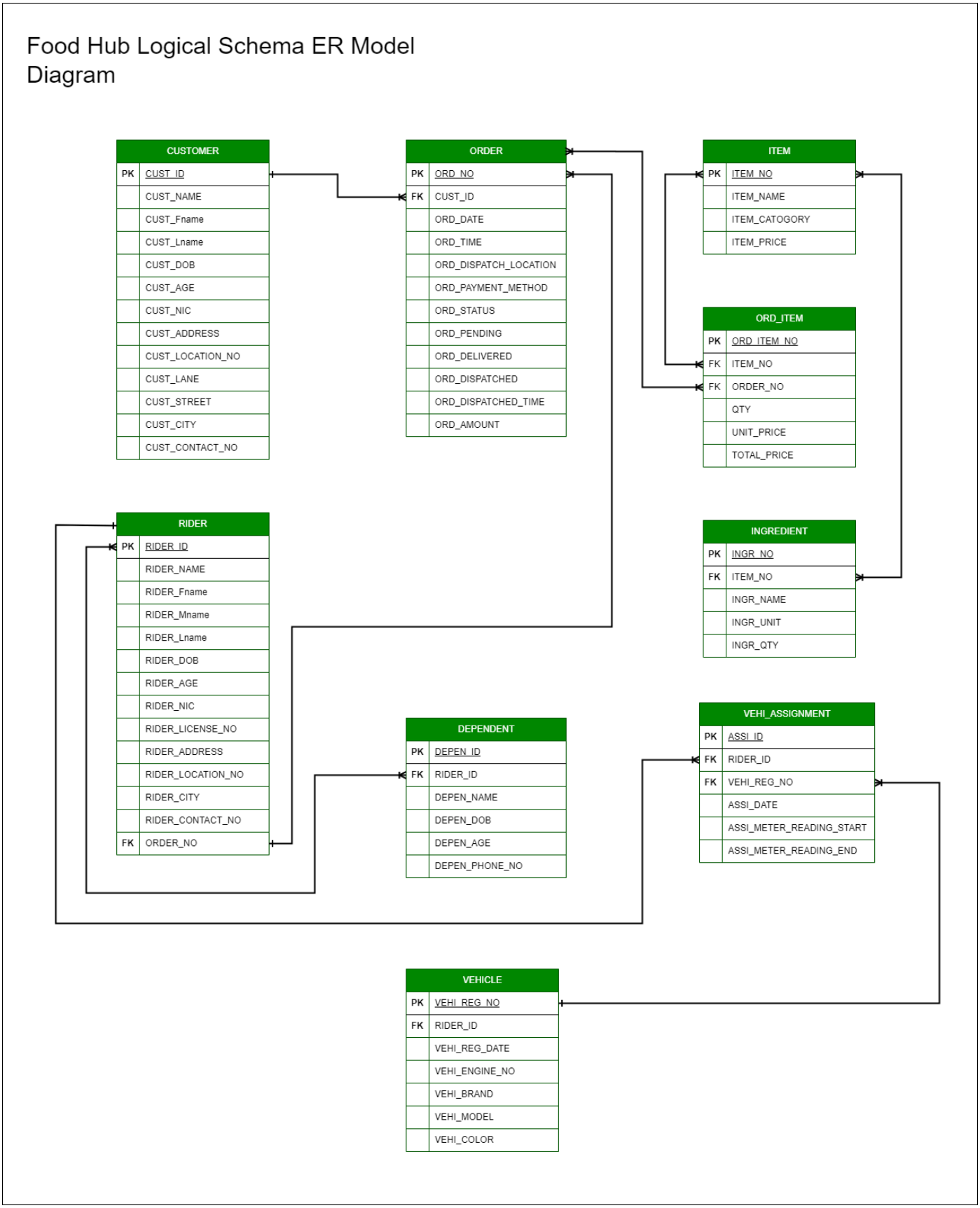


Figure 11. Boxed logical schema. Note ORDER_NO on RIDER and RIDER_ID on VEHICLE. Both are revised in the 2NF model. Windows and admin design baseline.

SECTION 16 · FIGURE 12

First normal form

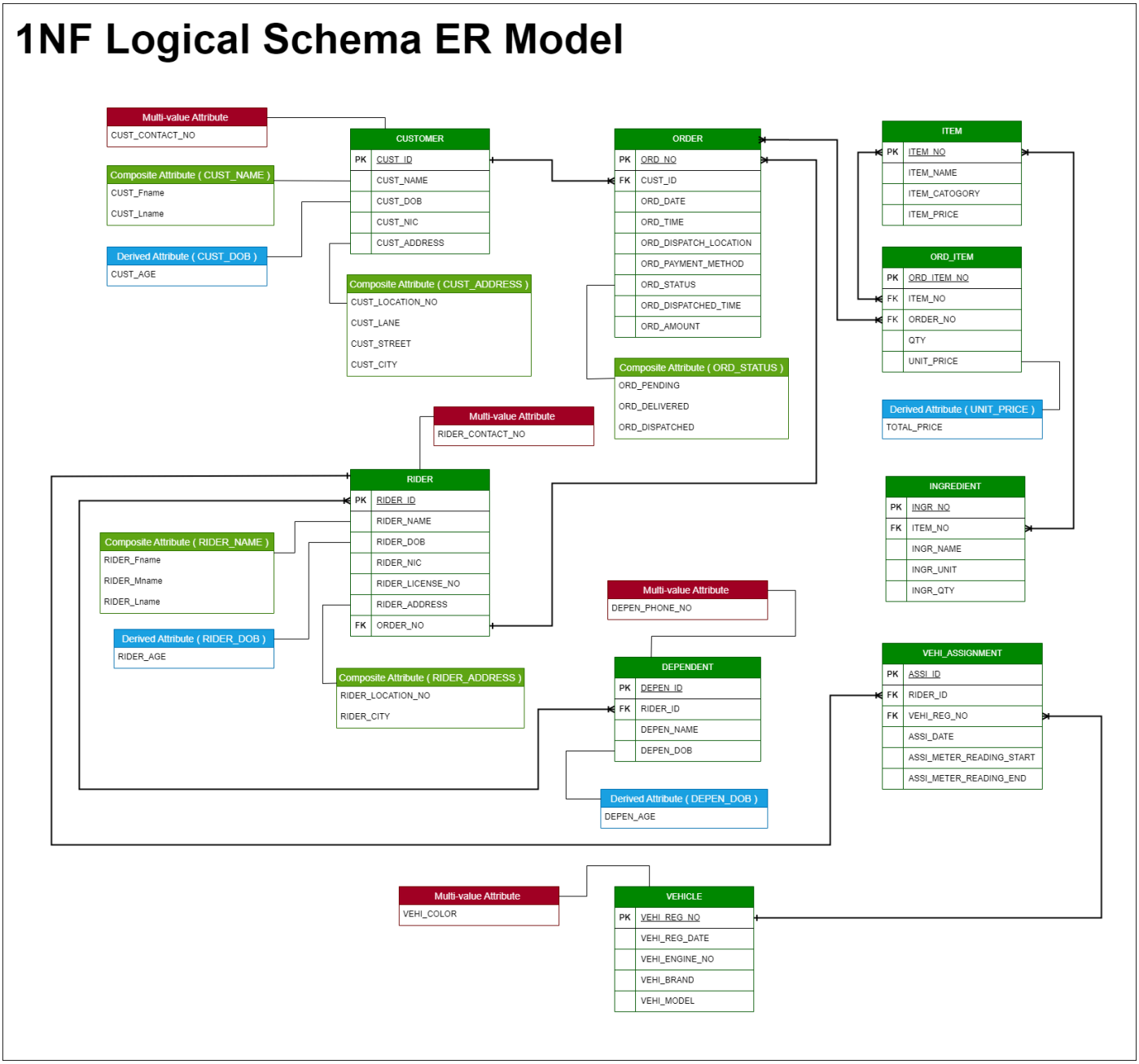


Figure 12. 1NF logical schema. Red callouts are multi-valued attributes, green callouts are composites, blue callouts are derived. The callouts are analysis marks. They are not yet all separate tables. Windows and admin design baseline.

SECTION 16

What 1NF is asking for

First normal form, in the sense used by this design set, asks every value to be atomic: no repeating groups inside a row, and no multi-valued cell. The drawing does the analysis by color rather than by finishing every new table. Red marks multi-valued attributes: CUST_CONTACT_NO, RIDER_CONTACT_NO, DEPEN_PHONE_NO, and VEHI_COLOR. Green marks composites: customer name, customer address, rider name, rider address, and order status (pending, delivered, dispatched). Blue marks derived attributes: customer age, rider age, dependent age, and total price from unit price and quantity.

A strict physical 1NF would already have replaced each red attribute with a child table, and would have replaced each green composite either with separate columns (which the main entity already partly has) or with a referenced address or person-name structure. The drawing keeps the composite boxes beside the entity. That is legitimate as a worksheet. It is not legitimate as a CREATE TABLE script. The implementation rule is:

- Store given name and family name in columns. Store a middle name for riders, because the class diagram has one. Do not also store a concatenated CUST_NAME as a second source of truth. A display name can be computed.
- Store location number, lane, street, and city as columns, or as one address row. Do not also store an unstructured CUST_ADDRESS unless it is clearly a leftover import field.
- Store each phone in a child row with a type (mobile, home, other). The prototype's single phone column is the temporary 1.0 shortcut.
- Do not store age. Store date of birth only if the business truly needs it. NIC and date of birth are sensitive. They are not required for a customer to order lunch, and they should not be copied onto the customer just because the rider model needs identity documents.
- Treat order status as one code, not as three flags plus a code. The green status composite is a warning that the earlier model repeated itself.

The 1NF figure still shows structural problems that are not 1NF problems. RIDER still has an ORDER_NO foreign key. INGREDIENT still has ITEM_NO. VEHICLE is tied through assignment but the surrounding model is about to change. Those are dependency problems for 2NF and 3NF, and the next two figures take them on. Leaving them visible on the 1NF sheet is useful. It shows the sequence rather than pretending the first step finished the design.

> **Category spelling.** Item category is written ITEM_CATOGORY on these drawings. The physical name should be category. Version 1.0 already did this better: a categories table, not a misspelled text column.

SECTION 16 · FIGURE 13

Second normal form

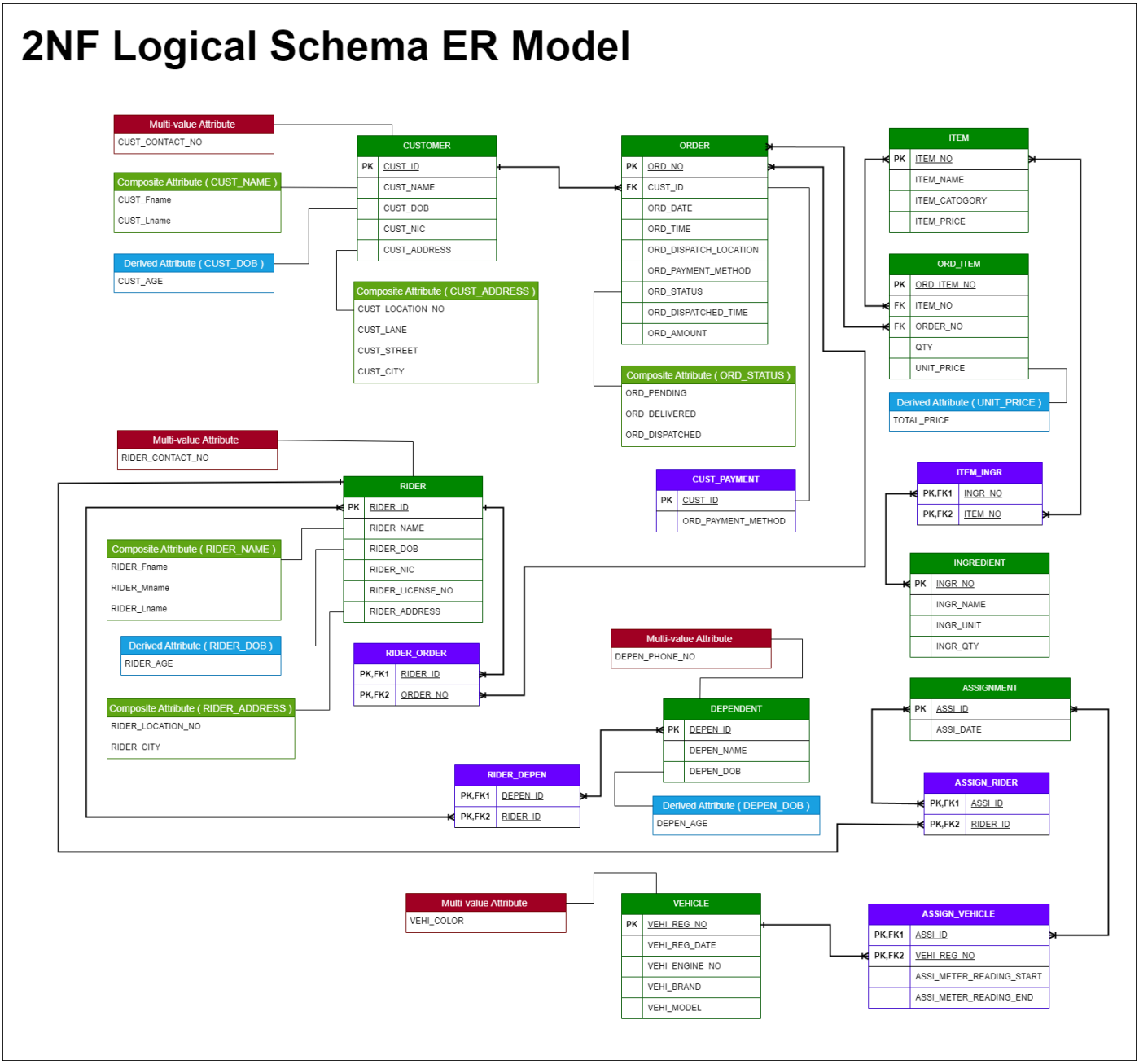


Figure 13. 2NF logical schema. New intersection entities are shown in a second color: CUST_PAYMENT, ITEM_INGR, RIDER_ORDER, RIDER_DEPEN, ASSIGN_RIDER, ASSIGN_VEHICLE. Windows and admin design baseline.

SECTION 16

What 2NF changes

Second normal form requires that every non-key attribute depend on the whole primary key, not on part of a composite key. The practical effect in this design is the introduction of intersection tables where the earlier model had stuffed a foreign key into the wrong parent.

New structure	Key	What it fixes
RIDER_ORDER	RIDER_ID + ORDER_NO	Removes ORDER_NO from RIDER. A rider can carry many orders. An order can record the rider who took it. If the business rule is exactly one rider per order, the intersection can be tightened with a unique constraint on ORDER_NO.
ITEM_INGR	INGR_NO + ITEM_NO	Replaces “ingredient belongs to one item.” Shared ingredients are one row in INGREDIENT, linked to many items. Quantity and unit still need a home. They belong on the link if they differ per recipe, and on the ingredient if they are intrinsic. Recipe quantity belongs on the link.
ASSIGNMENT split	ASSIGN_RIDER and ASSIGN_VEHICLE	An assignment event (ASSI_ID, date) connects to a rider and to a vehicle through link tables, instead of assuming the vehicle row itself is owned by one rider.
RIDER_DEPEN	DEPEN_ID + RIDER_ID	Drawn as a link. If a dependent has one rider, this table is redundant with a RIDER_ID column on DEPENDENT. Prefer the single foreign key unless dependents are shared.
CUST_PAYMENT	CUST_ID	Pulls payment method off the order and onto the customer. See the warning below. This is the weakest move on the sheet.

> **Payment method.** If a customer may pay one order in cash and the next by card, payment method is a fact about the order, or about a payment row under the order. Moving it to CUST_PAYMENT makes it a single preferred method. The use cases talk about orders, not about a wallet. Recommendation: keep a preferred method only as an optional customer column, and store the method actually used on the order. Do not adopt CUST_PAYMENT as the only place the fact exists.

The 2NF sheet still shows composites, multi-valued callouts, and derived ages beside the entities. Normalization of dependencies and atomicity are being tracked on the same page, which is why the colors remain. RIDER no longer carries the license number in the same way once 3NF extracts identity. Vehicle still carries brand and model directly. That is the transitive dependency the last sheet cuts.

Assignment meter readings sit on ASSIGN_VEHICLE in this drawing (start and end), while the assignment header carries the date. That can work if every assignment has one vehicle. It will feel awkward if a shift changes bikes. The shift flowchart reads as one vehicle per shift. A unique vehicle per assignment is the rule to encode. Meter columns can live on the assignment header itself if there is exactly one vehicle; the extra ASSIGN_VEHICLE table is only needed if the relationship is genuinely many-to-many. Given Figure 7, a simpler physical table is enough: assignment (id, rider_id, vehicle_reg_no, shift_start, shift_end, meter_start, meter_end). The 2NF split should be simplified in that direction unless a real case for two vehicles per assignment appears.

SECTION 16 · FIGURE 14

Third normal form

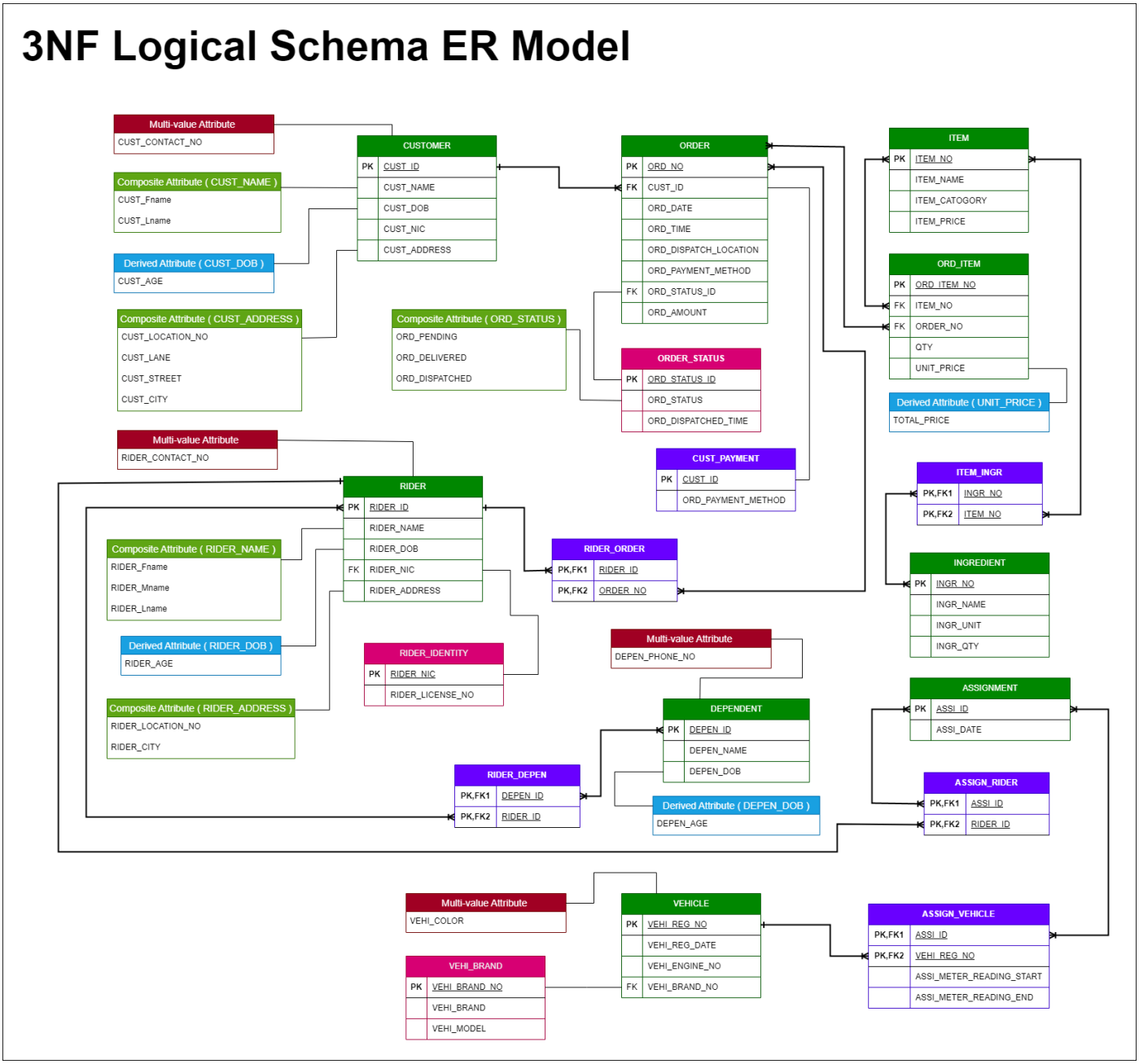


Figure 14. 3NF logical schema. New lookup-style entities: ORDER_STATUS, RIDER_IDENTITY, and VEHI_BRAND. This is the logical target for future development, after the corrections in the following section. Windows and admin design baseline.

SECTION 16

What 3NF changes, and what it still leaves

Third normal form requires that every non-key attribute depend on the key, the whole key, and nothing but the key. The drawing applies that idea in three new entities.

ORDER_STATUS. ORD_STATUS_ID identifies a status name. The order holds ORD_STATUS_ID instead of a bundle of flags. That matches the advice from the class diagram. The drawing also places ORD_DISPATCHED_TIME on the status entity. That does not match 3NF intent. Dispatch time is a fact about a particular order, or about an event, not a fact about the meaning of the word "Dispatched." If it lived on the lookup, every dispatched order would share one timestamp. Physical rule: the lookup holds the code and the label only. The order holds the current status id. A child table order_event holds status id, timestamp, and actor whenever a change happens. Dispatch time is the timestamp of the event whose status is Dispatched.

RIDER_IDENTITY. NIC and license number move off the rider's day-to-day row onto an identity entity keyed by NIC. The idea is right: a license is not a casual attribute to scatter through assignment screens, and it should not determine or be determined by a rider's address. The key choice is less right. Using NIC as the primary key makes a national identity number the join key throughout the database. Prefer a surrogate rider id, which the model already has, and a unique constraint on NIC. Store license number beside it, unique, and restrict the table to administrators. Riders should not need to read each other's NIC to do deliveries.

VEHI_BRAND. Brand and model leave the vehicle row. The vehicle keeps VEHI_BRAND_NO. This removes the repetition of brand text and the temptation to mistype it. The drawing stores brand and model in the same lookup row. That is acceptable if each row means "one model of one brand" (Honda · Dio, Honda · Shine). It is not acceptable if VEHI_BRAND means only the manufacturer and several models share it, because the model column would then be a partial fact. Name the table vehicle_model, with columns id, brand, and model, unique on (brand, model). Color stays a red multi-value callout. Unless bikes are recorded with multiple colors, collapse that callout to a single optional column and drop the multi-value claim.

Still not physical tables

Even on the 3NF sheet, phones and color remain callouts. A developer who stops at "we reached 3NF" and creates one column per callout has missed the red marks. The physical addition is small and should be done in the same release as the 3NF core:

- customer_phone (customer_id, phone, label)
- rider_phone (rider_id, phone, label)
- dependent_phone (dependent_id, phone, label)
- vehicle.color as a single column, unless multi-color is confirmed

Derived ages and line totals stay out of storage. The prototype's order_items table, with unit_price_minor and qty and no stored line total, is already the pattern to copy.

SECTION 16

Decisions before anyone codes the diagrams

The normalization sequence is good design work. It is not yet a migration script. The following decisions are the difference between implementing the drawings and implementing the system the drawings were reaching for.

#	Decision
1	Use the 3NF shape as the starting point, not Figure 10 or Figure 11. Those earlier sheets keep ORDER_NO on the rider and a rider foreign key culture on the vehicle.
2	Simplify assignment to one rider and one vehicle per shift row, with meter start and meter end on that row, unless a real double-vehicle shift appears.
3	Put payment method on the order. Keep a customer preference only as a convenience.
4	Put dispatch time on an order event, not on the status lookup.
5	Keep preparation status (the 1.0 list) and add delivery status (Dispatched, Delivered, Failed) instead of discarding either list.
6	Use ITEM_INGR for recipes. Store quantity per item on the link. Do not return to a single ITEM_NO on the ingredient.
7	Give dependents a rider foreign key, not a redundant link table, and decide the delete policy explicitly. Do not cascade personal data without a written reason.
8	Keep rider id as the key. Make NIC and license unique attributes of an identity row, visible to administrators only.
9	Do not add BRANCH until a multi-site decision is written. The conceptual ERD is not, by itself, that decision.
10	Do not store age, concatenated display name, or line total as authoritative columns.
11	Correct spellings in code: category, dispatch location, registration date, save. Do not copy drawing typos into the schema.
12	Separate customer profile richness from rider identity. Customers need a name, contacts, and a delivery address. They do not automatically need NIC because riders need NIC.

Version 1.0 already agrees with decisions 10 and 11 in spirit: minor-unit prices, a real categories table, snapshotted line prices, no age column. The future migration should extend that discipline rather than replace it with a literal dump of every oval.

SECTION 17

Gap analysis

The gap is the point of the prototype. Version 1.0 proved a channel. The drawings specify an operation. This table is the handover between them.

Future use case	Owner	Version 1.0	Gap
Place an order	Customer	Cart and transactional checkout	No dispatch location. No payment capture. No staff entry.
Enter customer and order details	Staff	Customer self-service only	No staff role and no counter order screen.
Request or view ingredients	Customer, staff	Description text only	No ingredient entity and no recipe link.
Receive order status	Customer	History, pills, notifications	Statuses are kitchen states, not dispatch and delivery.
Update order status	Staff	Admin status list, final states locked	Role is admin, not staff. No delivery outcome.
View assigned orders	Rider	None	No rider, no assignment.
Update delivery status	Rider	None	No Delivered or Failed, no reason, no delivery time.
Record dispatch time	Staff	None	No event table.
Record meter readings	Rider	None	No vehicle and no assignment meters.
Manage customers	Administrator	List, enable, disable, self-profile	No structured address, contacts, or staff-edited customer.
Manage items and ingredients	Administrator	Food CRUD, categories, images, stock	Ingredients absent. Stock is finished-item only.
Assign vehicles, manage vehicles and riders	Administrator	None	Entire D5 and D6 area.
Manage dependents	Administrator	None	Sensitive data. Needs a policy before a table.
Generate reports	Administrator	Four counts and monthly bars	No export, no rider or vehicle measures, audit log unused.

Read the last column as the backlog, not as a defect list against a finished logistics product. Against the prototype's own scope in Section 4, the implemented column is the completion record. Against the Windows and admin design, the gap column is the work that remains. Both readings are true, and they should be kept in separate sentences whenever the project is described in public. The repository badge may say completed. It may not say that riders and vehicles are live.

SECTION 18

Future development program

EasySoft's own statement of intent, in the source headers and the README, is to strengthen the architecture, improve function, and produce a more robust version. This section turns that sentence into a program. The program follows the drawings. It does not start by drawing new ones, and it does not start by coding every oval.

Release 1.1 — make the prototype maintainable

No new actor. Split `main.cpp` into authentication, catalog, orders, media, and a database unit. Split `App.vue` into views. Add Vue Router so menu, cart, orders, and each admin desk have URLs. Replace browser alerts and confirms. Introduce a migration tool and move the CREATE statements into a versioned migration. Write the audit log on status changes, price changes, and account activation, or remove the table. Add tests that fail if stock goes negative, if a customer calls an admin route, if a finalized order moves, and if a wrong password is accepted. Replace the process-wide mutex only after those tests exist. Move configuration — port, database path, upload directory, session lifetime, administrator code — into the environment. Stop documenting sample secrets.

Release 1.2 — customer and staff depth, still no rider

This release absorbs the parts of Figures 1, 3, and 4 that do not need a vehicle. Add a staff role created by an administrator, never by the public form. Staff can enter a customer and an order, which is the counter path on the flowchart. Customers keep self-service. Add structured address fields and a phone child table. Add ingredients and ITEM_INGR. Show ingredients on the food card, which closes "view ingredients" and "request order ingredients" in a read-only way. Put payment method on the order. If the business rule is pay-on-delivery, record cash or card as a tender type chosen at the counter or at the door, still without storing card numbers. Add order events so status changes have an actor and a time. Keep the 1.0 kitchen statuses.

Release 1.3 — dispatch, rider, vehicle

This is the Windows operational model arriving in the shared system: D5, D6, processes 2.0, 3.0, and 4.0, and Figures 5, 6, and 7. Administrator maintains riders and vehicles, including the vehicle-model lookup. A shift assignment binds one rider to one vehicle with meter start. Staff mark an order ready, then assign a rider, which writes RIDER_ORDER and, when the rider leaves, a dispatch event. The rider client lists only that rider's assignments, sets Delivered or Failed with a reason and a time, and at shift end writes meter end and releases the bike. Enforce one open assignment per vehicle. Compute distance. Notify the customer on dispatch and on the final delivery outcome, which finally makes the context diagram's "order dispatch confirmation" true.

SECTION 18

Future program, continued

Release 1.4 — administration that the use cases still lack

Dependents, with the delete policy decided in Section 16, and with phones in a child table. Identity documents behind administrator authorization. Formal reports: sales by period and category, order funnel, failed-delivery reasons, rider distance, vehicle utilization, low stock. Export to a file the administrator can keep. A report that cannot be tied to a store in the level-1 diagram should not be invented. In particular, do not report "branch performance" unless decision 9 is reversed and BRANCH is modeled.

Release 2.0 — a system that can leave the workstation

Version 2.0 is the first release allowed to be described as more than a prototype. It requires TLS in front of the service, secrets supplied at runtime, session cookies or an equivalent hardening of the bearer token, backups of the database, a release build rather than a Debug folder as the documented run, and a front-end build served with the API rather than a developer proxy. It requires the test suite from 1.1 to be green on the rider paths from 1.3. It is reasonable at that point to revisit SQLite. If riders and staff will hit the system concurrently from several networks, measure lock contention. Move to a server database only when the measurement says so, taking the migration scripts along. Do not rewrite SQL for sport.

Cross-cutting work that belongs in every release

- **Security.** Constant-time compare on every password check. Rate limiting on login. No administrator code on a public form. Least privilege for staff and rider tokens. Image storage that does not rely on a secret URL alone.
- **Privacy.** NIC, license, date of birth, and dependents are not menu data. Collect the minimum. Say why dependents exist before the screen exists — an emergency contact is a different data set from a family register, and it may be all the operation needs.
- **Interface.** Keep the FoodHub green system, Outfit and DM Sans, and the LKR minor-unit display. New desks should look like the 1.0 desk. Accessibility: form labels, which the prototype already uses, plus focus traps in dialogs and a real order-detail view.
- **Language.** The product is specified in English in this report and in the UI. A later Sinhala or Tamil interface is a product decision, not a schema change, if strings stay out of business keys.
- **Repository practice.** Commit each release's migration. Do not accumulate another year of work as a second "First Upload."

What success looks like

The program is finished enough to retire the phrase "prototype" when a customer can order, a staff member can correct that order, a rider can take a bike and close the delivery, an administrator can see the distance and the failures, and an engineer can change one module without reading a single 1,000-line file. The drawings will still be the map. They should be updated when a decision in Section 16 changes them — especially payment, status events, assignment, and branch — so the next report does not have to apologize for the same gaps.

SECTION 19

Risks, limitations, and conclusion

Limitations of version 1.0, stated plainly

- It is a workstation prototype. The documented run is Windows, Visual Studio 2026, vcpkg, Debug, and a Vite dev server.
- One C++ file and one Vue file will not absorb the rider model safely.
- There is no automated test suite and no CI.
- There is no real payment, no delivery fee, and no dispatch.
- Security is serious about passwords and casual about transport, session storage, and abuse.
- The audit table does not audit.
- The design set and the executable disagree, by decision. Anyone who skips Section 13 can misread the project.

Risks to the future program

Risk	Why it matters	Response already written
Literal implementation of every drawing	Would recreate contradictory keys, stored ages, and a shared dispatch timestamp	Section 16 decisions
Adding riders before tests and a split code base	Assignment bugs would land in an untestable file	Release 1.1 before 1.3
Treating dependents as just another CRUD screen	Personal data without a purpose	Policy before the table
Publishing the workstation admin code	Trivial administrator signup	Do not repeat it; replace it; remove public admin registration
Two schemas, desktop and web, drifting apart	The original problem returns	One corrected logical model, several clients
Branch scope appearing by accident	Half of the conceptual ERD is unused	Explicit decision required

Conclusion

FoodHub Version 1.0 is complete for the job it was given. EasySoft, through Evan Keal, took a C# .NET desktop concept, kept its meaning, and built a web prototype with Vue, a C++ Crow service, CMake, vcpkg, OpenSSL, and SQLite. The interface was designed with AI assistance. The system structure and the workflows were drawn by hand. Customers can order. Administrators can run the menu and the board. Passwords are hashed. Totals are exact integer minor units. The source is published.

The same project already knows what it is not. The use cases, data-flow diagrams, flowcharts, class diagram, and normalized logical models describe a delivery operation for the Windows application and its administrators: staff, riders, vehicles, meters, ingredients, dependents, and reports. Those drawings are the development baseline from this point forward. They should be implemented in the order in Section 18, with the corrections in Section 16, and not as a claim about the prototype that has just been closed.

> **FoodHub 1.0 — completed web application prototype.** Designed to evolve. Built to improve. EasySoft · Evan Keal · 2026.

References

Project source and README, repository github.com/aimstudios3001/FoodHub-Food-Order-System. Developer portfolio, evan-keal-portfolio.vercel.app. Draw.io figure set reproduced in this document. External technologies as used, not modified by this report: Vue.js, Vite, Crow, SQLite, OpenSSL, CMake, vcpkg, Visual Studio Code, Git, and diagrams.net.